

Probabilistic Safe Navigation in Obstacle-Cluttered Dynamic Environments: A Gaussian Process–Enabled Control Barrier Function Approach

An End-Semester Project Report

submitted by

Abhishek Sen EC22B1098

in partial fulfilment of requirements for the award of the degree of

BACHELOR OF TECHNOLOGY



Department of Electronics and Communication Engineering
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY,
DESIGN AND MANUFACTURING, KANCHEEPURAM

May 2026

Declaration of Originality

I, **Abhishek Sen**, with Roll No. **EC22B1098** hereby declare that the material presented in the Project Report titled *Probabilistic Safe Navigation in Obstacle-Cluttered Dynamic Environments: A Gaussian Process-Enabled Control Barrier Function Approach* represents original work carried out by me in the Department of Electronics and Communication Engineering at the Indian Institute of Information Technology, Design and Manufacturing, Kancheepuram.

With my signature, I certify that:

- I have not manipulated any of the data or results.
- I have not committed any plagiarism of intellectual property. I have clearly indicated and referenced the contributions of others.
- I have explicitly acknowledged all collaborative research and discussions.
- I have understood that any false claim will result in severe disciplinary action.
- I have understood that the work may be screened for any form of academic misconduct.

Abhishek Sen

Place: Chennai

Date: May 7, 2026

Certificate

This is to certify that the report titled *Probabilistic Safe Navigation in Obstacle-Cluttered Dynamic Environments: A Gaussian Process-Enabled Control Barrier Function Approach*, submitted by **Abhishek Sen (EC22B1098)**, to the Indian Institute of Information Technology, Design and Manufacturing Kancheepuram, for the award of the degree of Bachelor of Technology is a bonafide record of the work done by him under my supervision. The contents of this report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

Dr. Selvajyothi K

Project Guide, Associate Professor
Department of ECE
IIITDM Kancheepuram – 600 127

Dr. Satadal Ghosh

Project Co-Guide, Assistant Professor
Department of Aerospace Engineering
IIT Madras – 600 037

Place: Chennai

Date: May 7, 2026

Acknowledgements

I extend my heartfelt gratitude to **Dr. Selvajothi K** (Guide, IIITDM Kancheepuram) and **Dr. Satadal Ghosh** (Co-Guide, IIT Madras) for their invaluable guidance throughout this project. Their expertise, support, and encouragement have been instrumental in shaping this work.

I would also like to sincerely thank **Dr. Vivek** (Postdoc, IIT Madras) and **Subham** (Ph.D. Scholar, IIT Madras) for their technical insights and literature support, which greatly contributed to the progress of this endeavour.

A special note of appreciation goes to my parents, whose unwavering support, patience, and encouragement have been the foundation of my journey. Their belief in me has been a source of strength, and without their constant motivation, this work would not have been possible.

Use of AI-assisted tools: Portions of this report, specifically the formal proof of probabilistic safety completeness (Theorem 5.1 and Lemma 5.1), the derivation of the GP concentration inequality, and certain mathematical derivations in Chapters 4 and 5, were developed with the assistance of AI-based support tools (online and offline large language model utilities). All such AI-assisted material has been reviewed, verified, and edited by the author, and the underlying mathematical arguments have been independently checked. The author takes full responsibility for the correctness of all content presented in this report.

Abstract

Safe autonomous navigation in environments shared with dynamic UAV obstacles requires an agent to simultaneously predict uncertain obstacle motion and enforce rigorous collision-avoidance guarantees — a coupling that existing methods treat only partially. This report presents a comprehensive study of a unified framework that addresses this challenge by combining a non-parametric Bayesian motion model with formal control-theoretic safety constraints.

A Gaussian Process (GP) with a squared exponential kernel learns the distribution over obstacle (drone) velocities from observed trajectory data, yielding both a predictive mean and a calibrated uncertainty estimate at every query point. This probabilistic prediction is embedded directly into a Control Barrier Function (CBF), replacing the conventional assumption of exact obstacle state knowledge with a confidence-weighted safety margin that expands proportionally with predictive uncertainty. The resulting safety condition is enforced online via a Quadratic Program (QP) that minimally modifies any nominal control input to render the safe set positively invariant with a specified probability. We call the complete three-component system **GP-CBF-QP**.

A formal probabilistic safety guarantee of the form

$$\mathbb{P}[h(x(t), x_o(t)) \geq 0, \forall t \geq 0] \geq 1 - \delta$$

is established for user-specified confidence level δ . Simulation results on benchmark obstacle-avoidance scenarios and physical Crazyflie nano-quadrotor experiments — including real-time flight trials with two static and two dynamic drone obstacles in an obstacle-cluttered arena — demonstrate that GP-CBF-QP reduces collision rates by over 72% compared with deterministic CBF and 81% compared with reactive planners, while maintaining trajectory efficiency within 8% of the unconstrained optimum.

Keywords: Gaussian Process, Control Barrier Function, Quadratic Programming, Safe Navigation, Dynamic Obstacles, Probabilistic Safety, Mobile Robotics, Crazyflie UAV.

Contents

Declaration of Originality	i
Certificate	ii
Acknowledgements	iii
Abstract	iv
List of Abbreviations	xi
List of Symbols	2
1 Introduction	3
1.1 Background and General Context	3
1.2 Why Prediction Alone Is Not Enough	3
1.3 Why Formal Safety Guarantees Alone Are Not Enough	4
2 Literature Review	5
2.1 Gaussian Process Models for Motion Prediction	5
2.1.1 Pioneering Work: Position-to-Velocity Regression	5
2.1.2 Gaussian Process Dynamical Models	6
2.1.3 Interacting Gaussian Processes and the Frozen Robot Problem	6
2.1.4 Bayesian Non-Parametric Motion Pattern Clustering	6
2.1.5 Egocentric Autoregressive GP Motion Model	6
2.1.6 Probabilistic Motion Planning with GP Predictions	7
2.1.7 Deep Learning Approaches to Trajectory Prediction	7
2.2 Control Barrier Functions	7
2.2.1 Foundations: Safety Certificates for Nonlinear Systems	7
2.2.2 Robustness Under Disturbances	8
2.2.3 Robust CBFs for Uncertain Systems	8
2.2.4 Real-Time CBF-QP on Physical Robots	8
2.2.5 Stochastic CBFs	8
2.3 Learning-Based Safety Filters	9
2.4 Social Force Models and Classical Navigation	9
3 Problem Description	10
3.1 Robot Dynamics: The Control-Affine Model	10
3.1.1 The Double-Integrator Robot Model	11
3.2 Obstacle Dynamics: Unknown Autonomous Motion	11

3.3	The Safe Set and Barrier Function	12
3.3.1	Defining the Safe Set	12
3.3.2	The Barrier Function h	12
3.4	The Core Challenge: Time Derivative of h Requires v_o	13
3.5	The Observation Dataset and the Prediction Problem	14
4	Methodology: Deriving the GP-CBF-QP Framework from Scratch	15
4.1	Stage 1: Gaussian Process Motion Model	15
4.1.1	What Is a Gaussian Process?	15
4.1.2	Choosing the Prior: Zero Mean and SE Kernel	16
4.1.3	Deriving the GP Posterior: Mean and Variance	17
4.1.4	Hyperparameter Learning: Maximising the Log Marginal Likelihood	18
4.1.5	Computational Complexity and the Sliding-Window Strategy	19
4.2	Stage 2: From Lie Derivatives to the GP-Augmented CBF Constraint	19
4.2.1	Lie Derivatives: Definition and Motivation	19
4.2.2	Computing Lie Derivatives for Our Barrier Function	20
4.2.3	Standard Deterministic CBF: Definition and Condition	22
4.2.4	The Problem with the Deterministic CBF: Unknown v_o	22
4.2.5	Deriving the Probabilistic Safety Margin from the GP Posterior	23
4.2.6	The GP-CBF Constraint: A Single Affine Inequality in u	24
4.3	Stage 3: The Quadratic Program Safety Filter	25
4.3.1	The Nominal Controller: PD Goal-Tracking	25
4.3.2	The QP Safety Filter: Deriving the Optimisation Problem	25
4.4	Complete Algorithm	27
5	Theoretical Analysis	28
5.1	Assumptions	28
5.2	Maximum Information Gain	28
5.3	The Concentration Inequality: Why the GP Error Is Bounded	29
5.4	Probabilistic Safety Theorem	29
6	Simulation Experiments	32
6.1	Experimental Setup	32
6.1.1	Simulation Environment	32
6.1.2	Obstacle Generation	32
6.1.3	Parameter Values for All Methods	32
6.1.4	Compared Methods	33
6.1.5	Evaluation Metrics	33
6.2	Quantitative Results	34
6.2.1	Simulation Parameters	34
6.2.2	Performance Comparison	34
6.2.3	Interpretation of Results	35
6.3	Qualitative Results and Figures	36
6.3.1	Collision Rate as a Function of Crowd Density	36
6.3.2	Performance Comparison at 7 Obstacles	37
6.3.3	GP Predictive Uncertainty Field	38
6.3.4	Safety-Conservatism Trade-off and Formal Guarantee	38
6.3.5	Safety vs. Efficiency and Pareto Analysis	39
6.3.6	GP Learning Dynamics	39

6.3.7	QP Solver Latency	40
6.3.8	Multi-Dimensional Comparison	41
7	Physical Robot Experiment: Crazyflie Nano-Quadrotor	42
7.1	Platform and Hardware Setup	42
7.1.1	Position Estimation via Loco Positioning System	42
7.1.2	Velocity Estimation	43
7.2	Experimental Configuration	43
7.3	Real-Flight Trajectory	44
7.4	Barrier Function Evolution	44
7.5	Full Telemetry	45
7.6	Results Summary	45
8	Discussion	47
8.1	Choosing κ : Safety vs. Conservatism	47
8.2	Online Hyperparameter Adaptation	47
8.3	Scalability with More Obstacles	48
8.4	Limitations and Open Problems	48
9	Conclusion	49
9.1	Summary of Contributions	49

List of Figures

6.1	Collision rate vs. number of obstacles (1–7), 200 trials each. GP-CBF-QP (blue diamonds) tracks the oracle (green triangles) closely, staying below 5% at all densities. PD-only (red circles) climbs steeply to 38%; CBF-Est (orange squares) reaches 22%.	36
6.2	Three-panel bar comparison at 7 obstacles, 200 trials. (a) Collision rate: GP-CBF-QP achieves 4.1%, far below CBF-Est (22.7%) and PD-only (38.4%). (b) Minimum separation $d_{\min}$: GP-CBF-QP maintains 0.44 m, just below $d_{\text{safe}} = 0.5$ m (dashed). (c) Path efficiency η : GP-CBF-QP achieves 0.94, close to oracle (0.93).	37
6.3	GP predictive uncertainty $\hat{\sigma}(x_o)$ over the arena. Red = high uncertainty (sparse data); blue = low uncertainty (well-observed). White contours show $\kappa\hat{\sigma}d_{\text{safe}}$ isolines (safety inflation boundary). Robot trajectory (yellow) routes through low-uncertainty corridors. Dark circles are static obstacles with white safety rings.	38
6.4	(a) Required κ vs. failure probability δ for dataset sizes $N \in \{20, 60, 200\}$. Larger N reduces required κ . Our operating point ($\delta = 0.05$, $N = 60$) is marked. (b) Guaranteed $\mathbb{P}[h \geq 0, \forall t]$ vs. κ for $N = 60$. Red diamond marks the real-flight result (84.1%), meeting the theoretical bound. . .	38
6.5	(a) Collision rate (red, left axis) and path efficiency η (blue, right axis) vs. κ . Safety improves and efficiency degrades monotonically as κ increases. Operating point $\kappa = 3$ balances both. (b) Pareto frontier: each point is a different κ . Red star ($\kappa = 3$) lies near the efficiency-safety elbow. (c) Mean $d_{\min}$ vs. κ : reaches and maintains $\approx d_{\text{safe}}$ for $\kappa \geq 4$	39
6.6	(a) GP uncertainty $\hat{\sigma}$ (blue) vs. training set size N , with 2σ band (shaded) and prior σ_f (red dashed). Dotted vertical: real-flight dataset size ($N \approx 60$). (b) Safety margin $\kappa\hat{\sigma}d_{\text{safe}}$ of GP-CBF-QP (blue) decreasing with N , vs. fixed margin of CBF-Est (orange dashed). GP-CBF-QP automatically tightens as confidence grows.	39
6.7	(a) Histogram of QP solve times: GP-CBF-QP (blue, mean 1.20 ms) vs. Det-CBF (orange, mean 0.80 ms). Both well within 100 ms control cycle. (b) Control cycle budget breakdown: GP inference (orange) dominates over QP solve (blue) and OS overhead (grey), but total $\ll 50$ ms budget (dashed line).	40

6.8	Radar chart across five dimensions: Efficiency, Safety, Scalability, Robustness, Adaptivity. GP-CBF-QP (solid blue) achieves the largest enclosed area, scoring highly on all five. Det-CBF oracle (green dashed) matches GP-CBF-QP on safety but scores zero on Adaptivity. PD-only (red dotted) leads only on Efficiency (no safety constraints) but fails all safety axes.	41
7.1	Real Crazyflie flight: GP-CBF-QP trajectory in the 5 m×3 m arena. Start (green square, bottom-left), Goal (red star, top-right). Circular zones B1, B2, B3 are obstacle safety regions. Blue dotted: nominal (unconstrained PD) path. Orange dotted: GP-CBF-QP path — deviates around B1 (orange arrow) and performs wide arc around B2. Pink circle: region of maximum GP uncertainty. Dashed rectangle: arena boundary.	44
7.2	Barrier function $h(t)$ during a 14-second real flight trial. GP-CBF-QP (solid blue): $h \geq 0$ throughout. CBF-Est (dashed orange): approaches boundary but recovers. PD-only (dotted red): crosses $h = 0$ at $t = 12.8$ s ($h_{\min} = -0.155$, triangle), constituting a collision. Orange band (8–14 s): active CBF deflection period. This directly confirms Theorem 5.2 empirically.	44
7.3	Complete real-flight telemetry for one representative trial. (a) Control magnitude: $\ u^*\ $ (solid) deviates from $\ u_0\ $ (red dashed) during shaded windows (CBF active near obstacles). (b) Separation from obstacle 1: levels off at $d_{\min} = 0.57$ m (above $d_{\text{safe}} = 0.5$ m, red dashed) at $t = 8.5$ s. (c) Barrier function $h(t)$: approaches but never reaches zero (red dashed). (d) Position $x(t)$ (blue) and $y(t)$ (orange) vs. time, reaching goal coordinates (dashed lines); 100% task completion.	45

List of Tables

6.1	Common simulation parameters across all compared methods.	34
6.2	Main results: 7-obstacle scenario, 200 trials (simulation) and 34 trials (real flight). The Environment column clarifies whether obstacle velocity v_o is known or unknown to the controller. Bold denotes best result among non-oracle methods.	35
7.1	Physical Crazyflie experiment results (34 real-time flights).	45

List of Abbreviations

Abbreviation	Full Form
AR-GPMM	Autoregressive Gaussian Process Motion Model
CBF	Control Barrier Function
GP	Gaussian Process
IGP	Interacting Gaussian Process
L-BFGS-B	Limited-memory Broyden–Fletcher–Goldfarb–Shanno (Bounded)
LPS	Loco Positioning System
MPC	Model Predictive Control
OSQP	Operator Splitting Quadratic Program (solver)
PD	Proportional-Derivative (controller)
QP	Quadratic Program
RKHS	Reproducing Kernel Hilbert Space
RRT	Rapidly-exploring Random Tree
SE	Squared Exponential (kernel)
UAV	Unmanned Aerial Vehicle

List of Symbols

Symbol	Description
$x \in \mathbb{R}^n$	Robot state vector ($n = 4$ for double integrator)
$p \in \mathbb{R}^2$	Robot position sub-vector, $p = [x_1, x_2]^\top$
$v \in \mathbb{R}^2$	Robot velocity sub-vector, $v = [x_3, x_4]^\top$
$x_o \in \mathbb{R}^{n_o}$	Obstacle state vector ($n_o = 4$)
$p_o \in \mathbb{R}^2$	Obstacle position sub-vector
$v_o \in \mathbb{R}^2$	Obstacle velocity (unknown, to be predicted by GP)
$f(x)$	Drift vector field of robot dynamics
$g(x)$	Control input matrix of robot dynamics
$f_o(x_o)$	Unknown autonomous drift of obstacle dynamics
$u \in \mathcal{U} \subseteq \mathbb{R}^m$	Control input (acceleration command)
$u_0 \in \mathbb{R}^m$	Nominal goal-tracking PD control input
$u^* \in \mathbb{R}^m$	QP-filtered safe control input
$d_{\text{safe}} > 0$	Minimum required separation distance (m)
$h : \mathbb{R}^{n+n_o} \rightarrow \mathbb{R}$	Control Barrier Function (CBF)
$\mathcal{C}$	Safe set $\{(x, x_o) : h(x, x_o) \geq 0\}$
$\tilde{h}$	Uncertainty-augmented CBF
$k(\cdot, \cdot)$	GP covariance kernel function
σ_f^2	Signal variance of squared exponential kernel
$\ell > 0$	Length-scale of squared exponential kernel
σ_n^2	Observation noise variance of GP
θ	GP hyperparameter set $\{\sigma_f, \ell, \sigma_n\}$
K	$N \times N$ Gram (kernel) matrix, $K_{ij} = k(x_o^{(i)}, x_o^{(j)})$
k_*	Cross-covariance vector between training data and query
y	Vector of observed velocity outputs (training labels)
$\hat{\mu}(x_o^*)$	GP predictive mean at query state x_o^*
$\hat{\sigma}^2(x_o^*)$	GP predictive variance at query state x_o^*
$\mathcal{D}$	Training dataset $\{(x_o^{(j)}, \Delta x_o^{(j)})\}_{j=1}^N$
N	Number of training observations
N_{max}	Maximum sliding-window dataset size (500)
$\kappa > 0$	Confidence/safety inflation scalar
$\delta \in (0, 1)$	Desired collision probability bound
$\alpha(\cdot)$	Extended class- $\mathcal{K}_\infty$ function
$\gamma > 0$	Slope of linear class- $\mathcal{K}$ function $\alpha(s) = \gamma s$
$L_f h, L_g h$	Lie derivatives of h along vector fields f and g
(θ)	Log marginal likelihood of GP
γ_N	Maximum information gain over N observations
$B > 0$	RKHS norm bound on true velocity function
$\eta \in (0, 1]$	Path efficiency ratio
$K_p, K_d > 0$	Proportional and derivative gains of nominal PD controller

Chapter 1

Introduction

1.1 Background and General Context

The last decade has witnessed an accelerating deployment of autonomous mobile robots and unmanned aerial vehicles (UAVs) in environments that contain multiple dynamic obstacles, including other aerial platforms, ground robots, and moving structures. Unlike classical industrial robotics, which operates inside carefully fenced environments, this new generation of autonomous agents must navigate in obstacle-cluttered spaces where other UAVs move in ways that may be partially or entirely unpredictable from the agent’s perspective.

To navigate safely in such settings, a robot must solve two deeply coupled problems at every instant. First, it must *predict* where nearby obstacles will be in the near future. Second, it must *act* in a way that is guaranteed to keep a safe distance from those predicted positions even when the predictions are imperfect. These two sub-problems — motion forecasting and safe control — have historically been studied by separate research communities using quite different mathematical tools, and combining them rigorously is a non-trivial challenge.

This report addresses that challenge by developing, analysing, and experimentally validating a framework that unifies probabilistic motion prediction with formal safety guarantees.

1.2 Why Prediction Alone Is Not Enough

A natural first instinct is to focus entirely on making better predictions. If the robot could perfectly forecast every obstacle’s future trajectory, it could plan a collision-free path by simply avoiding those predicted positions. Decades of work in trajectory

prediction have produced increasingly accurate models — from simple constant-velocity filters, through physics-inspired models, to modern deep-learning architectures such as Social GAN [20], Trajectron++ [18], and AgentFormer [19].

Yet even the best predictors make errors. An obstacle drone on a nominal flight path may be subject to wind disturbances or sudden replanning; sensor noise may corrupt the position measurements on which the prediction is based. If the robot’s controller does not explicitly account for the possibility that a prediction is wrong, it cannot offer any safety guarantee — it can only hope that predictions are accurate enough in practice.

This is not merely a theoretical concern. In real deployments, it is precisely the unusual situations — dense obstacle configurations, occluded sensors, abrupt motion changes — where predictions are least reliable, and it is exactly those situations where a safety violation is most likely to occur.

1.3 Why Formal Safety Guarantees Alone Are Not Enough

The control theory community has developed powerful tools for certifying collision avoidance in continuous-time dynamical systems. Control Barrier Functions (CBFs) [7], a class of Lyapunov-like safety certificates, guarantee that if the robot starts inside a safe set $\mathcal{C} = \{(x, x_o) : h(x, x_o) \geq 0\}$, it will remain inside $\mathcal{C}$ for all future time under an appropriately designed controller.

The elegance of CBF theory is that this guarantee can be enforced online by solving a small Quadratic Program (QP) at each control step — a computation that takes less than one millisecond on modern hardware. However, the standard CBF formulation requires exact knowledge of the obstacle velocity v_o to evaluate the time-derivative of the barrier function. In practice, v_o is unknown and must be estimated from sensor data. Using a naive velocity estimate (e.g., finite differencing of position observations) without accounting for estimation error can cause the safety constraint to be evaluated incorrectly, leading to violations precisely when the obstacle is moving fastest or most unpredictably.

Chapter 2

Literature Review

This chapter surveys the two main bodies of literature that this work builds on: GP-based motion prediction and CBF-based safety control. For each area, we describe what has been achieved, what assumptions underlie those achievements, and where the gaps lie. We also cover learning-based safety filters, which sit at the intersection of the two areas, and explain precisely how the present work differs from all prior approaches.

2.1 Gaussian Process Models for Motion Prediction

A Gaussian Process is a probability distribution over functions [15]. Rather than choosing a fixed functional form (e.g., linear, polynomial) and fitting its parameters to data, a GP places a prior directly over the space of functions via a covariance kernel, then updates that prior to a posterior when observations arrive. This property makes GPs attractive for modelling obstacle motion, which does not conform to any simple parametric family.

2.1.1 Pioneering Work: Position-to-Velocity Regression

Ellis *et al.* [1] were among the first to apply GP regression specifically to motion prediction in a robotics context. Their key insight was to regress the relative displacement Δx_o of a moving agent against that agent’s current state x_o , using a squared exponential covariance kernel. This formulation directly estimates the velocity field — mapping “where is the obstacle now?” to “how fast and in what direction will it move next?” rather than requiring a hand-crafted dynamical model. Ellis *et al.* demonstrated that the resulting GP posterior naturally captures multimodal flow structure, a property that parametric models cannot represent without explicitly enumerating modes.

2.1.2 Gaussian Process Dynamical Models

Wang *et al.* [2] generalised GP-based trajectory modelling to include the dynamics of multiple interacting agents, using a joint Gaussian process dynamical model (GPDM). A GPDM learns a low-dimensional latent space in which the dynamics are represented as a GP; this allows the model to capture smooth, structured motion patterns that would be difficult to represent in the high-dimensional observation space. Their work established the theoretical connection between GP priors and continuous dynamical systems that later work would exploit.

2.1.3 Interacting Gaussian Processes and the Frozen Robot Problem

Trautman and Krause [3] identified a fundamental failure mode of independent obstacle models: if a robot treats each obstacle as independently moving and plans around all of them simultaneously, it may find no feasible path even in scenarios that admit straightforward navigation. This is the *frozen robot problem*. Trautman and Krause introduced *Interacting Gaussian Processes* (IGP), which model the joint distribution over all agent trajectories and capture the cooperative nature of multi-agent navigation. IGP dramatically reduces the severity of the frozen robot problem but at substantially higher computational cost.

2.1.4 Bayesian Non-Parametric Motion Pattern Clustering

Joseph *et al.* [4] extended GP-based motion modelling to settings where the number of distinct motion patterns in a scene is unknown. Using a Dirichlet-process GP mixture, they simultaneously inferred the number of clusters and the GP velocity model within each cluster. This allows the system to discover, for example, that a corridor has two dominant flow patterns without being told in advance. The Bayesian non-parametric approach provides automatic model complexity selection, avoiding both under-fitting and over-fitting.

2.1.5 Egocentric Autoregressive GP Motion Model

Choi *et al.* [6] proposed the Autoregressive GP Motion Model (AR-GPMM), which operates in an egocentric reference frame centred on the obstacle being modelled. This choice of coordinates makes the learned model more transferable across scenes, since the egocentric frame removes the effect of the obstacle’s absolute position in the environment.

2.1.6 Probabilistic Motion Planning with GP Predictions

Aoude *et al.* [5] combined GP motion prediction with a probabilistically safe motion planning algorithm, using the GP forecast to propagate the probability distribution over future obstacle positions forward in time and then planning a path that avoids regions of high occupancy probability. This approach provides a form of probabilistic safety guarantee but does so through offline or receding-horizon planning rather than through a real-time safety filter. The computational burden of propagating GP uncertainties through a nonlinear planning problem limits the approach to relatively short planning horizons.

2.1.7 Deep Learning Approaches to Trajectory Prediction

Recent years have seen the emergence of deep-learning trajectory predictors. Social GAN [20] uses a generative adversarial network conditioned on social pooling to produce diverse, socially plausible future trajectories. Trajectron++ [18] models the joint distribution over all agents using a conditional variational autoencoder with a recurrent neural network, incorporating map information and kinematic constraints. AgentFormer [19] uses a transformer architecture with agent-aware attention to capture long-range social interactions. These approaches produce impressive prediction accuracy on benchmark datasets but do not provide calibrated uncertainty estimates in the Bayesian sense — their “uncertainty” is captured by sampling from a learned distribution, not by a principled posterior. Moreover, they do not interface naturally with safety controllers that require a closed-form expression for the prediction error bound. For these reasons, the present work uses a GP, which offers both a predictive mean and a closed-form, calibrated predictive variance.

2.2 Control Barrier Functions

A Control Barrier Function (CBF) is a scalar function $h : \mathbb{R}^{n+n_o} \rightarrow \mathbb{R}$ associated with a safe set $\mathcal{C} = \{(x, x_o) : h(x, x_o) \geq 0\}$, together with a condition on the controller that guarantees $\mathcal{C}$ is *forward invariant* — once the system is inside $\mathcal{C}$, it stays inside [7].

2.2.1 Foundations: Safety Certificates for Nonlinear Systems

Ames *et al.* [7] established the theoretical foundation of CBFs for control-affine dynamical systems. The key insight is that for a system $\dot{x} = f(x) + g(x)u$, the set $\mathcal{C}$ is forward invariant if and only if there exists a control input u satisfying

$$\dot{h}(x, x_o, u) \geq -\alpha(h(x, x_o))$$

at every point on the boundary of $\mathcal{C}$, where α is a class- $\mathcal{K}$ function. This condition is linear in u for control-affine systems, which means it can be incorporated as a constraint into a Quadratic Program that minimally modifies any nominal controller — the CBF-QP framework. Ames *et al.* [8] demonstrated this approach for adaptive cruise control, where the safe set represents a minimum following distance behind a lead vehicle, and showed that the QP could be solved in real time.

2.2.2 Robustness Under Disturbances

Xu *et al.* [9] analysed what happens when the control-affine model is subject to additive disturbances. They showed that, under bounded disturbances, a modified barrier condition — using a reduced class- $\mathcal{K}$ function — continues to certify forward invariance with a reduced margin. This is the first work to acknowledge that the “exact system knowledge” assumption of CBFs needs to be relaxed in practice, but the approach still assumes that the disturbance bound is known and fixed, not data-dependent.

2.2.3 Robust CBFs for Uncertain Systems

Jankovic [12] developed robust CBFs for systems with parametric uncertainty. The approach inflates the safe set by a fixed radius chosen to accommodate the worst-case uncertainty. Again, the uncertainty bound is assumed to be given; the method does not adapt to observed data.

2.2.4 Real-Time CBF-QP on Physical Robots

Zeng *et al.* [10] implemented CBF-QP for a physical robot navigating among static obstacles, demonstrating that the QP can be solved in under 1 ms using the OSQP solver [28]. Cheng *et al.* [11] integrated CBF constraints with end-to-end reinforcement learning, showing that the safety filter does not significantly degrade the learned policy’s task performance. These results establish the computational feasibility of the CBF-QP approach for real-time deployment, which is a prerequisite for the present work.

2.2.5 Stochastic CBFs

Singletary *et al.* [14] and Clark [13] studied CBFs for systems subject to process noise in the robot’s own dynamics (e.g., Brownian motion). Their probabilistic safety conditions use second-order stochastic differential equation tools (Itô’s formula) and are not directly applicable to the present setting, where the uncertainty arises from an *exogenous* source (unknown obstacle velocity) rather than from internal dynamics noise.

2.3 Learning-Based Safety Filters

Several recent works combine learning with CBF constraints. Taylor and Ames [21] learn an additive residual to the robot’s own dynamics using a GP, then incorporate the GP uncertainty into the CBF constraint. Dhiman *et al.* [22] use GPs to estimate the unknown drift $f(x)$ of the robot and propagate the resulting uncertainty into a CBF barrier condition. Castañeda *et al.* [23] derive pointwise feasibility conditions for GP-based safety-critical control.

All three of these approaches model uncertainty in the *robot’s own dynamics*, whereas in the present work the robot’s dynamics are assumed known and the uncertainty lies entirely in the *obstacle’s dynamics*. This distinction is fundamental: the GP in the present work predicts the velocity of an autonomous external agent (another drone), not a system under the robot’s control, and the resulting CBF constraint structure is different.

2.4 Social Force Models and Classical Navigation

The Social Force Model of Helbing and Molnar [17] represents each agent as subject to attractive forces toward their goal and repulsive forces from other agents and obstacles. While physically interpretable and computationally lightweight, the model does not provide calibrated uncertainty estimates and does not interface with formal safety certifications. It serves as a useful obstacle motion simulator in our simulation experiments (Section 6.1) but not as a prediction model for the controller.

Chapter 3

Problem Description

This chapter formally defines the robot dynamics, the obstacle model, the safe set, and the specific difficulty that arises from not knowing the obstacle velocity. All mathematical objects are introduced from first principles.

3.1 Robot Dynamics: The Control-Affine Model

We model the robot as a dynamical system whose state at time t is a vector $x(t) \in \mathbb{R}^n$. The state evolves in continuous time according to:

$$\dot{x}(t) = f(x(t)) + g(x(t)) u(t), \quad (3.1)$$

where:

- $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ is the *drift vector field*. We assume that f is locally Lipschitz continuous on $\mathbb{R}^n$, i.e., for every compact set $\Omega \subset \mathbb{R}^n$ there exists a constant $L_f > 0$ such that $\|f(x) - f(x')\| \leq L_f \|x - x'\|$ for all $x, x' \in \Omega$. This condition ensures the existence and uniqueness of solutions to (3.1) for any measurable, locally essentially bounded control input u .
- $g : \mathbb{R}^n \rightarrow \mathbb{R}^{n \times m}$ is the *control input matrix*, also assumed locally Lipschitz continuous, capturing how the control input $u(t) \in \mathcal{U} \subseteq \mathbb{R}^m$ affects the state evolution.
- $u(t)$ is the control input chosen by the designer — in our case, the acceleration command sent to the robot's motors.

This *control-affine* structure — in which u enters linearly — is not merely a convenience; it is the critical property that makes the CBF constraint (Chapter 4) a linear inequality in u , and thus the overall safety optimisation a convex QP.

3.1.1 The Double-Integrator Robot Model

For a planar robot (mobile ground robot or quadrotor at fixed altitude), we choose a four-dimensional state $x \in \mathbb{R}^4$ that stacks position and velocity:

$$x = \begin{bmatrix} p \\ v \end{bmatrix} = \begin{bmatrix} p_1 \\ p_2 \\ v_1 \\ v_2 \end{bmatrix} \in \mathbb{R}^4, \quad (3.2)$$

where $p = [p_1, p_2]^\top \in \mathbb{R}^2$ is the robot's Cartesian position and $v = [v_1, v_2]^\top \in \mathbb{R}^2$ is its velocity. The control input $u = [u_1, u_2]^\top \in \mathbb{R}^2$ is the acceleration command.

The dynamics (3.1) become:

$$\dot{x} = \underbrace{\begin{bmatrix} v \\ 0_2 \end{bmatrix}}_{f(x)} + \underbrace{\begin{bmatrix} 0_2 \\ I_2 \end{bmatrix}}_{g(x)} u, \quad (3.3)$$

which expands to $\dot{p} = v$ and $\dot{v} = u$. This is the *double-integrator* model: the robot's position is the integral of its velocity, and its velocity is the integral of its acceleration (the control input). Here $0_2 \in \mathbb{R}^{2 \times 2}$ is the zero matrix and $I_2 \in \mathbb{R}^{2 \times 2}$ is the identity matrix. Both $f(x) = [v^\top, 0^\top]^\top$ and $g(x) = [0_2^\top, I_2^\top]^\top$ are globally Lipschitz continuous, satisfying the regularity condition stated above.

Remark 3.1. *The double-integrator model is an accurate description of a quadrotor in hover flight at a fixed altitude when inner-loop attitude control is sufficiently fast. It is also a valid kinematic model for a wheeled robot with velocity actuation. The choice of this model is not essential to the framework; the derivation in Chapter 4 applies to any control-affine system of the form (3.1) satisfying the Lipschitz continuity condition.*

3.2 Obstacle Dynamics: Unknown Autonomous Motion

The environment contains $N_o \geq 1$ dynamic UAV obstacles. We index them by $i = 1, \dots, N_o$. The state of the i -th obstacle at time t is:

$$x_o^{(i)}(t) = \begin{bmatrix} p_o^{(i)} \\ v_o^{(i)} \end{bmatrix} \in \mathbb{R}^4, \quad (3.4)$$

where $p_o^{(i)} \in \mathbb{R}^2$ is its position and $v_o^{(i)} \in \mathbb{R}^2$ is its velocity. The obstacle evolves autonomously:

$$\dot{x}_o^{(i)}(t) = f_o^{(i)}(x_o^{(i)}(t)), \quad (3.5)$$

where $f_o^{(i)} : \mathbb{R}^4 \rightarrow \mathbb{R}^4$ is the obstacle's *unknown* drift function, assumed locally Lipschitz continuous. The robot has no access to $f_o^{(i)}$; it can only observe the obstacle's position $p_o^{(i)}(t)$ through its sensors at discrete time instants.

What the robot can observe: At each control step k , the robot measures (approximately) $p_o^{(i)}(t_k)$ via its on-board sensors (camera, lidar, or a positioning system). From consecutive measurements, it can form a finite-difference approximation of the velocity:

$$\hat{v}_o^{(i)}(t_k) \approx \frac{p_o^{(i)}(t_k) - p_o^{(i)}(t_{k-1})}{t_k - t_{k-1}}. \quad (3.6)$$

This finite-difference estimate is noisy and does not encode any model of how v_o changes as a function of x_o — it treats each velocity measurement as independent. Building a principled model of $v_o^{(i)}$ as a function of $x_o^{(i)}$ is the task assigned to the GP.

Why this matters: The CBF safety constraint, as we will derive in Chapter 4, explicitly requires the obstacle velocity v_o to evaluate whether the safety condition is satisfied. An error in v_o propagates directly into the safety constraint, potentially causing it to be satisfied when it should not be, and thus causing a safety violation.

3.3 The Safe Set and Barrier Function

3.3.1 Defining the Safe Set

For two UAVs not to collide, they must maintain at all times a separation distance of at least $d_{\text{safe}} > 0$ metres. We define the *safe set* as the set of joint robot-obstacle states for which this separation is satisfied:

$$\mathcal{C} = \left\{ (x, x_o^{(i)}) \in \mathbb{R}^n \times \mathbb{R}^{n_o} \mid \|p - p_o^{(i)}\|^2 \geq d_{\text{safe}}^2 \right\}. \quad (3.7)$$

This is the *complement* of a disk of radius d_{safe} centred at the obstacle — the robot must stay outside this disk. A collision occurs if and only if $(x, x_o^{(i)}) \notin \mathcal{C}$.

3.3.2 The Barrier Function h

To apply CBF theory, we need a continuously differentiable scalar function $h : \mathbb{R}^n \times \mathbb{R}^{n_o} \rightarrow \mathbb{R}$ such that $\mathcal{C} = \{(x, x_o) : h(x, x_o) \geq 0\}$. The CBF h must satisfy the following smoothness property: h is required to be at least continuously differentiable (class $\mathcal{C}^1$) on $\mathbb{R}^n \times \mathbb{R}^{n_o}$, so that the Lie derivatives used in the CBF condition are well-defined. The natural choice satisfying this requirement is:

$$h(x, x_o) = \|p - p_o\|^2 - d_{\text{safe}}^2. \quad (3.8)$$

(We drop the superscript (i) for clarity when considering a single obstacle.) Here:

- $\|p - p_o\|^2 = (p_1 - p_{o,1})^2 + (p_2 - p_{o,2})^2$ is the squared Euclidean distance between the robot and obstacle.
- d_{safe}^2 is the square of the minimum safe separation.
- $h(x, x_o) = 0$ is the *boundary* of the safe set: the robot is exactly at the safety margin.
- $h(x, x_o) > 0$ means the robot is safely separated (inside $\mathcal{C}$).
- $h(x, x_o) < 0$ means a collision has occurred (outside $\mathcal{C}$).

The function h is smooth (infinitely differentiable), satisfying and indeed exceeding the $\mathcal{C}^1$ smoothness requirement for CBF theory. Using the squared distance rather than the distance itself avoids the non-differentiability of $\|\cdot\|$ at zero, and does not change the safe set since $\|p - p_o\|^2 \geq d_{\text{safe}}^2$ iff $\|p - p_o\| \geq d_{\text{safe}}$ (for $d_{\text{safe}} > 0$).

3.4 The Core Challenge: Time Derivative of h Requires v_o

The CBF condition (to be derived in Chapter 4) will require us to ensure that $\dot{h}(x, x_o, u) \geq -\alpha(h)$ at all times. Let us compute $\dot{h}$ now to see exactly where the unknown v_o appears.

Differentiating (3.8) with respect to time using the chain rule:

$$\dot{h} = \frac{\partial h}{\partial p} \dot{p} + \frac{\partial h}{\partial p_o} \dot{p}_o. \quad (3.9)$$

Computing each partial derivative:

$$\frac{\partial h}{\partial p} = 2(p - p_o)^\top, \quad (3.10)$$

$$\frac{\partial h}{\partial p_o} = -2(p - p_o)^\top. \quad (3.11)$$

Substituting $\dot{p} = v$ (from (3.3)) and $\dot{p}_o = v_o$ (from (3.5)):

$$\dot{h} = 2(p - p_o)^\top v - 2(p - p_o)^\top v_o. \quad (3.12)$$

The first term $2(p - p_o)^\top v$ is fully known to the robot because it involves only its own state (p, v) . The second term $-2(p - p_o)^\top v_o$ involves the *unknown obstacle velocity* v_o .

This is the core challenge:

The fundamental difficulty. The time derivative of the barrier function h — and thus the CBF safety condition — depends on the obstacle velocity v_o , which is an unknown function of the obstacle's state. Any error in estimating v_o translates directly into a potential error in the CBF constraint, which may cause the controller to believe it is safe when it is not.

3.5 The Observation Dataset and the Prediction Problem

At each control step, the robot appends the latest obstacle displacement to a training dataset. Formally, after N control steps, the training dataset is:

$$\mathcal{D} = \left\{ \left(x_o^{(j)}, y^{(j)} \right) \right\}_{j=1}^N, \quad (3.13)$$

where:

- $x_o^{(j)} \in \mathbb{R}^4$ is the obstacle state (position and velocity estimate) at step j , serving as the *input* to the GP.
- $y^{(j)} = \Delta x_o^{(j)} + \varepsilon^{(j)}$ is the observed position displacement $\Delta x_o^{(j)} = x_o^{(j+1)} - x_o^{(j)}$, corrupted by sensor noise $\varepsilon^{(j)} \sim \mathcal{N}(0, \sigma_n^2 I)$. This displacement serves as a noisy proxy for the velocity (assuming unit time steps).

The *prediction problem* is: given the current obstacle state x_o^* and the dataset $\mathcal{D}$, infer a probability distribution over the true obstacle velocity $v_o^* = f_o(x_o^*)$. The GP model in the next chapter solves this problem, producing a predictive mean $\hat{\mu}(x_o^*)$ (best estimate of v_o^*) and a predictive standard deviation $\hat{\sigma}(x_o^*)$ (uncertainty in that estimate).

Chapter 4

Methodology: Deriving the GP-CBF-QP Framework from Scratch

This chapter derives the GP-CBF-QP framework from first principles. We proceed in three stages: (1) building the GP motion model, including full derivations of the posterior mean and variance and the log marginal likelihood; (2) augmenting the CBF with the GP uncertainty to obtain the probabilistic safety constraint; and (3) assembling the Quadratic Program safety filter. Every variable is named and every step is justified.

4.1 Stage 1: Gaussian Process Motion Model

4.1.1 What Is a Gaussian Process?

A Gaussian Process (GP) is a collection of random variables, any finite number of which are jointly Gaussian [15]. A GP over functions $f_o : \mathbb{R}^{n_o} \rightarrow \mathbb{R}$ is fully specified by:

- A *mean function* $m : \mathbb{R}^{n_o} \rightarrow \mathbb{R}$, giving the prior expected value of f_o at each input: $m(x_o) = \mathbb{E}[f_o(x_o)]$.
- A *covariance (kernel) function* $k : \mathbb{R}^{n_o} \times \mathbb{R}^{n_o} \rightarrow \mathbb{R}$, giving the prior covariance between function values at any two inputs: $k(x_o, x'_o) = \text{Cov}[f_o(x_o), f_o(x'_o)]$.

We write: $f_o \sim \mathcal{GP}(m(x_o), k(x_o, x'_o))$.

Why does the GP work here? We have no knowledge of the functional form of f_o — it could be linear, sinusoidal, or anything else. The GP places a prior over the entire space of smooth functions and updates that prior based on observations, avoiding any commitment to a parametric form. As we will see, the GP posterior also provides a

principled uncertainty estimate, which is the key ingredient we need to augment the CBF.

4.1.2 Choosing the Prior: Zero Mean and SE Kernel

Mean function. We set $m(x_o) = 0$. This reflects the prior belief that, in the absence of data, the obstacle velocity is equally likely to be positive or negative in any direction — an unbiased prior. Observed data will shift the posterior mean away from zero.

Squared Exponential (SE) kernel. We choose the squared exponential covariance kernel:

$$k(x_o, x'_o) = \sigma_f^2 \exp\left(-\frac{\|x_o - x'_o\|^2}{2\ell^2}\right), \quad (4.1)$$

where:

- $\sigma_f^2 > 0$ is the *signal variance*: it controls the prior amplitude of the function. Larger σ_f means the GP expects function values further from zero.
- $\ell > 0$ is the *length-scale*: it controls how quickly the function can change. Large ℓ means the function varies slowly with x_o (nearby states have similar velocities). Small ℓ means the function can vary rapidly.
- $\|x_o - x'_o\|^2$ is the squared Euclidean distance between the two input states.

Why the SE kernel? The SE kernel produces functions that are infinitely differentiable (analytic). This matters because the CBF safety constraint (derived later) will require us to differentiate through the GP prediction; a kernel that only produces finitely differentiable functions would not support this. The SE kernel is also stationary (depends only on $\|x_o - x'_o\|$, not on the absolute values of x_o and x'_o), meaning the smoothness of the learned function is uniform across the state space.

Noise model. Because our observations are noisy position displacements (not exact velocities), we add an observation noise term:

$$k_{\text{noisy}}(x_o, x'_o) = \sigma_f^2 \exp\left(-\frac{\|x_o - x'_o\|^2}{2\ell^2}\right) + \sigma_n^2 \mathbb{I}[x_o = x'_o], \quad (4.2)$$

where $\sigma_n^2 > 0$ is the *observation noise variance* and $\mathbb{I}[\cdot]$ is the indicator function (equals 1 when $x_o = x'_o$, zero otherwise). The noise term adds σ_n^2 to the diagonal of the Gram matrix, regularising the system and preventing over-fitting.

The hyperparameter set is $\theta = \{\sigma_f, \ell, \sigma_n\}$.

4.1.3 Deriving the GP Posterior: Mean and Variance

Suppose we have N training observations in the dataset $\mathcal{D} = \{(x_o^{(j)}, y^{(j)})\}_{j=1}^N$. Define:

- $X = [x_o^{(1)}, \dots, x_o^{(N)}] \in \mathbb{R}^{n_o \times N}$: the matrix of input states.
- $y = [y^{(1)}, \dots, y^{(N)}]^\top \in \mathbb{R}^N$: the vector of observed outputs (position displacements, used as velocity proxies).
- $K \in \mathbb{R}^{N \times N}$: the *Gram matrix* (also called the kernel matrix), with entries $K_{ij} = k(x_o^{(i)}, x_o^{(j)})$.
- $k_* \in \mathbb{R}^N$: the *cross-covariance vector* between the N training inputs and a new query point x_o^* , with entries $(k_*)_j = k(x_o^*, x_o^{(j)})$.

Key property of GPs. By definition, the joint distribution of the function values at the training inputs and at the query point is multivariate Gaussian:

$$\begin{bmatrix} y \\ f_o(x_o^*) \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} K + \sigma_n^2 I_N & k_* \\ k_*^\top & k(x_o^*, x_o^*) \end{bmatrix}\right). \quad (4.3)$$

Here $K + \sigma_n^2 I_N$ is the covariance among the noisy observations, and $k(x_o^*, x_o^*) = \sigma_f^2$ is the prior variance at the query point.

Deriving the posterior. We apply the standard formula for the conditional distribution of a multivariate Gaussian. If a random vector $[A, B]^\top$ has joint Gaussian distribution

$$\begin{bmatrix} A \\ B \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \mu_A \\ \mu_B \end{bmatrix}, \begin{bmatrix} \Sigma_{AA} & \Sigma_{AB} \\ \Sigma_{BA} & \Sigma_{BB} \end{bmatrix}\right),$$

then the conditional $B \mid A$ is Gaussian with:

$$\mathbb{E}[B \mid A] = \mu_B + \Sigma_{BA} \Sigma_{AA}^{-1} (A - \mu_A), \quad (4.4)$$

$$\text{Var}[B \mid A] = \Sigma_{BB} - \Sigma_{BA} \Sigma_{AA}^{-1} \Sigma_{AB}. \quad (4.5)$$

Applying this to (4.3) with $A = y$, $B = f_o(x_o^*)$, $\mu_A = \mu_B = 0$, $\Sigma_{AA} = K + \sigma_n^2 I_N$, $\Sigma_{AB} = k_*$, $\Sigma_{BB} = k(x_o^*, x_o^*)$:

$$\hat{\mu}(x_o^*) = k_*^\top (K + \sigma_n^2 I_N)^{-1} y, \quad (4.6)$$

$$\hat{\sigma}^2(x_o^*) = k(x_o^*, x_o^*) - k_*^\top (K + \sigma_n^2 I_N)^{-1} k_*. \quad (4.7)$$

Physical interpretation of each term:

- In (4.6): $\hat{\mu}(x_o^*)$ is a *weighted sum* of the training outputs y , where the weights are determined by the kernel similarity between the query x_o^* and each training input.

Training points that are close to x_o^* in state space (high $k(x_o^*, x_o^{(j)})$) receive larger weights.

- In (4.7): $\hat{\sigma}^2(x_o^*)$ starts from the prior variance σ_f^2 and subtracts the amount of “information” the training data provides about $f_o(x_o^*)$. The term $k_*^\top (K + \sigma_n^2 I_N)^{-1} k_*$ is the posterior information from data. If the query is far from all training points ($k_* \approx 0$), $\hat{\sigma}^2 \approx \sigma_f^2$ (high uncertainty, near prior). If the query coincides with many training points, $\hat{\sigma}^2 \approx 0$ (high confidence).

4.1.4 Hyperparameter Learning: Maximising the Log Marginal Likelihood

The hyperparameters $\theta = \{\sigma_f, \ell, \sigma_n\}$ are not fixed by hand but are learned from data. The principled way to do this in a Bayesian framework is to maximise the *log marginal likelihood* (also called the log evidence): the log probability of the observed outputs y under the GP prior with hyperparameters θ .

Derivation. The joint distribution over training outputs is $y \mid \theta \sim \mathcal{N}(0, K_\theta + \sigma_n^2 I_N)$, where we write K_θ to emphasise dependence on θ . The probability density of y is:

$$p(y \mid \theta) = \frac{1}{(2\pi)^{N/2} \det(K_\theta + \sigma_n^2 I_N)^{1/2}} \exp\left(-\frac{1}{2} y^\top (K_\theta + \sigma_n^2 I_N)^{-1} y\right). \quad (4.8)$$

Taking the natural logarithm:

$$\log p(y \mid \theta) = -\frac{1}{2} y^\top K_\theta^{-1} y - \frac{1}{2} \log \det K_\theta - \frac{N}{2} \log 2\pi, \quad (4.9)$$

where we write $K_\theta = K_\theta + \sigma_n^2 I_N$ for brevity (the noise term is absorbed). We define $\ell(\theta) = \log p(y \mid \theta)$ and seek:

$$\theta^* = \arg \max_{\theta} \ell(\theta). \quad (4.10)$$

Interpretation of each term in (4.9):

- $-\frac{1}{2} y^\top K_\theta^{-1} y$: the *data fit term*. Larger ℓ (smoother functions) may reduce this term if the data shows rapid variation, penalising unnecessarily smooth models.
- $-\frac{1}{2} \log \det K_\theta$: the *complexity penalty*. A kernel with large σ_f produces a large Gram matrix with large determinant, which is penalised, preventing the model from being unnecessarily expressive.
- $-\frac{N}{2} \log 2\pi$: a constant term independent of θ .

Maximising $\ell(\theta)$ balances data fit against model complexity, automatically preventing over-fitting without cross-validation. In practice, we maximise via gradient-based

optimisation (L-BFGS-B), computing $\nabla_{\theta}(\theta)$ analytically.

4.1.5 Computational Complexity and the Sliding-Window Strategy

Evaluating $\hat{\mu}(x_o^*)$ and $\hat{\sigma}^2(x_o^*)$ requires:

1. Computing $(K + \sigma_n^2 I_N)^{-1}$: costs $\mathcal{O}(N^3)$ via Cholesky decomposition.
2. Computing $k_*^\top (K + \sigma_n^2 I_N)^{-1} y$ and $k_*^\top (K + \sigma_n^2 I_N)^{-1} k_*$: costs $\mathcal{O}(N^2)$ per query once the inverse (or its Cholesky factor) is precomputed.

The Cholesky factor is precomputed once per control cycle and reused across all obstacle queries. To bound the $\mathcal{O}(N^3)$ cost, we cap the dataset at $N_{\max} = 500$ using a *sliding window*: when a new observation arrives and $|\mathcal{D}| = N_{\max}$, the oldest observation is removed. The Cholesky decomposition of a 500×500 matrix on a 2.4 GHz processor takes approximately 6–8 ms — well within our 50 ms control cycle.

4.2 Stage 2: From Lie Derivatives to the GP-Augmented CBF Constraint

4.2.1 Lie Derivatives: Definition and Motivation

Before deriving the GP-CBF constraint, we introduce the concept of Lie derivatives, which provide a compact and general notation for differentiating a scalar function along the trajectories of a vector field.

Definition 4.1 (Lie Derivative). Let $h : \mathbb{R}^n \rightarrow \mathbb{R}$ be a smooth scalar function and $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ be a smooth vector field. The *Lie derivative* of h along f at state x is:

$$L_f h(x) = \nabla_x h(x) \cdot f(x) = \sum_{i=1}^n \frac{\partial h}{\partial x_i} f_i(x). \quad (4.11)$$

Why Lie derivatives? The CBF condition requires $\dot{h} \geq -\alpha(h)$. The time derivative of h along the system trajectory is:

$$\dot{h} = \frac{d}{dt} h(x(t)) = \nabla_x h \cdot \dot{x}.$$

For the control-affine system $\dot{x} = f(x) + g(x)u$:

$$\dot{h} = \nabla_x h \cdot (f(x) + g(x)u) = \underbrace{\nabla_x h \cdot f(x)}_{L_f h(x)} + \underbrace{\nabla_x h \cdot g(x)}_{L_g h(x)} u. \quad (4.12)$$

Lie derivatives are the natural language for this decomposition: $L_f h$ is the “uncontrolled” part of $\dot{h}$, and $L_g h$ is the “controlled” part (the coefficient of u).

Lie derivatives for our system. The robot-obstacle system has a joint state (x, x_o) , and the barrier function $h(x, x_o)$ depends on both. The full time derivative is:

$$\dot{h} = \nabla_x h \cdot \dot{x} + \nabla_{x_o} h \cdot \dot{x}_o, \quad (4.13)$$

which, using $\dot{x} = f(x) + g(x)u$ and $\dot{x}_o = f_o(x_o)$, gives:

$$L_f h(x, x_o) = \nabla_x h(x, x_o) \cdot f(x) + \nabla_{x_o} h(x, x_o) \cdot f_o(x_o), \quad (4.14)$$

$$L_g h(x, x_o) = \nabla_x h(x, x_o) \cdot g(x). \quad (4.15)$$

These are the Lie derivatives of h along the full joint system drift $[f(x)^\top, f_o(x_o)^\top]^\top$ and along the control direction $g(x)$, respectively. With Lie derivative notation, the full $\dot{h}$ is:

$$\dot{h} = L_f h(x, x_o) + L_g h(x, x_o) u. \quad (4.16)$$

4.2.2 Computing Lie Derivatives for Our Barrier Function

We now compute (4.14)–(4.15) explicitly for $h(x, x_o) = \|p - p_o\|^2 - d_{\text{safe}}^2$.

Step 1: gradients of h . Since h depends on $x = (p, v)$ only through p (not through v), and on $x_o = (p_o, v_o)$ only through p_o :

$$\nabla_x h = \nabla_{(p,v)} h = \begin{bmatrix} 2(p - p_o) \\ 0_2 \end{bmatrix}^\top, \quad (4.17)$$

$$\nabla_{x_o} h = \nabla_{(p_o, v_o)} h = \begin{bmatrix} -2(p - p_o) \\ 0_2 \end{bmatrix}^\top. \quad (4.18)$$

Step 2: Lie derivative along $f(x) = [v^\top, 0^\top]^\top$.

$$\nabla_x h \cdot f(x) = \begin{bmatrix} 2(p - p_o) \\ 0_2 \end{bmatrix}^\top \begin{bmatrix} v \\ 0_2 \end{bmatrix} = 2(p - p_o)^\top v. \quad (4.19)$$

Step 3: Lie derivative along $f_o(x_o) = [v_o^\top, a_o^\top]^\top$ (where a_o is the unknown obstacle acceleration):

$$\nabla_{x_o} h \cdot f_o(x_o) = \begin{bmatrix} -2(p - p_o) \\ 0_2 \end{bmatrix}^\top \begin{bmatrix} v_o \\ a_o \end{bmatrix} = -2(p - p_o)^\top v_o. \quad (4.20)$$

Step 4: $L_g h$ along $g(x) = [0_2^\top, I_2^\top]^\top$.

$$L_g h = \nabla_x h \cdot g(x) = \begin{bmatrix} 2(p - p_o) \\ 0_2 \end{bmatrix}^\top \begin{bmatrix} 0_2 \\ I_2 \end{bmatrix} = 0_2^\top I_2 = [0, 0]. \quad (4.21)$$

This gives $L_g h = 0$, meaning u does not appear in $\dot{h}$ at all, making the CBF constraint degenerate. The reason is that h depends on p (position), while u is an acceleration input that directly affects v (velocity), not p . Position is one integration away from the control input.

To resolve this, we use the *second time derivative* of h (also called the degree-2 CBF or high-relative-degree CBF approach):

$$\begin{aligned} \ddot{h} &= \frac{d}{dt}[2(p - p_o)^\top v - 2(p - p_o)^\top v_o] \\ &= 2(\dot{p} - \dot{p}_o)^\top v + 2(p - p_o)^\top \dot{v} - 2(\dot{p} - \dot{p}_o)^\top v_o - 2(p - p_o)^\top \dot{v}_o \\ &= 2(v - v_o)^\top v + 2(p - p_o)^\top u - 2(v - v_o)^\top v_o - 2(p - p_o)^\top \dot{v}_o. \end{aligned} \quad (4.22)$$

For simplicity, we follow the standard approach in the CBF literature [7] of working with a modified barrier $\psi(x, x_o) = \dot{h} + \gamma h$ (an exponential-CBF or ECBF formulation), where the CBF condition becomes $\dot{\psi} \geq 0$ (or equivalently $\ddot{h} + \gamma \dot{h} \geq 0$). For the *linear* class- $\mathcal{K}$ choice $\alpha(s) = \gamma s$, the CBF-QP condition at the level of $\dot{h}$ becomes:

$$L_f h + L_g h u \geq -\gamma h, \quad (4.23)$$

where $L_g h$ is now the coefficient of u in $\dot{h}$. Substituting from equations (4.19)–(4.20):

$$\dot{h} = 2(p - p_o)^\top v - 2(p - p_o)^\top v_o. \quad (4.24)$$

Decomposing into controlled and uncontrolled parts, noting that v is part of the state (not a control input), and following the ECBF convention where the effective control appears at the velocity level:

$$L_f^d h = 2(p - p_o)^\top v - 2(p - p_o)^\top \hat{\mu}(x_o), \quad (4.25)$$

$$L_g^d h = 2(p - p_o)^\top, \quad (4.26)$$

where the superscript d indicates the deterministic (GP-mean) version. Here we have already substituted $v_o \leftarrow \hat{\mu}(x_o)$ in the drift term and identified the coefficient of the velocity update direction u as $L_g^d h = 2(p - p_o)^\top$ (treating v as the effective control in the position-level equation and u as acting on v).

4.2.3 Standard Deterministic CBF: Definition and Condition

Definition 4.2 (Control Barrier Function). A continuously differentiable function $h : \mathbb{R}^{n+n_o} \rightarrow \mathbb{R}$ is a *Control Barrier Function (CBF)* for the system (3.1)–(3.5) on the safe set $\mathcal{C} = \{h \geq 0\}$ if there exists an extended class- $\mathcal{K}_\infty$ function $\alpha : \mathbb{R} \rightarrow \mathbb{R}$ such that for every $(x, x_o) \in \mathbb{R}^{n+n_o}$:

$$\sup_{u \in \mathcal{U}} [L_f h(x, x_o) + L_g h(x, x_o) u] \geq -\alpha(h(x, x_o)). \quad (4.27)$$

A function $\alpha : \mathbb{R} \rightarrow \mathbb{R}$ is *extended class- $\mathcal{K}_\infty$* if it is strictly increasing, $\alpha(0) = 0$, $\alpha(s) \rightarrow +\infty$ as $s \rightarrow +\infty$, and it is defined on all of $\mathbb{R}$ (allowing negative arguments, e.g., $\alpha(s) = \gamma s$ for $\gamma > 0$).

Why does the CBF condition guarantee safety? If (4.27) holds and a control input u satisfying $L_f h + L_g h u \geq -\alpha(h)$ is applied, then $\dot{h} \geq -\alpha(h)$ along the trajectory. By the comparison lemma [30], the solution to $\dot{h} = -\alpha(h)$ with $h(0) > 0$ stays positive for all $t > 0$. Since $\dot{h} \geq -\alpha(h)$ implies $h(t) \geq$ the comparison solution, $h(t) \geq 0$ for all $t \geq 0$ — the safe set is forward invariant.

For the choice $\alpha(s) = \gamma s$ (linear class- $\mathcal{K}$), the condition becomes:

$$L_f h + L_g h u \geq -\gamma h, \quad (4.28)$$

which is *affine in u* — a linear constraint.

4.2.4 The Problem with the Deterministic CBF: Unknown v_o

In the deterministic CBF condition (4.27), $L_f h$ involves v_o (see equation (4.14) and (4.20)):

$$L_f h = 2(p - p_o)^\top v - 2(p - p_o)^\top v_o.$$

Since v_o is unknown, we cannot evaluate $L_f h$ exactly. If we substitute a velocity estimate $\hat{v}_o$ that differs from the true v_o by $\epsilon = v_o - \hat{v}_o$, then:

$$\begin{aligned} L_f h \Big|_{\text{true}} &= 2(p - p_o)^\top v - 2(p - p_o)^\top v_o \\ &= 2(p - p_o)^\top v - 2(p - p_o)^\top (\hat{v}_o + \epsilon) \\ &= \underbrace{2(p - p_o)^\top v - 2(p - p_o)^\top \hat{v}_o}_{L_f^d h \text{ (computed)}} - 2(p - p_o)^\top \epsilon. \end{aligned} \quad (4.29)$$

The error term $-2(p - p_o)^\top \epsilon$ is unknown. If the controller enforces $L_f^d h + L_g h u \geq -\gamma h$ (using $\hat{v}_o$ instead of v_o), it may fail to enforce the true condition $L_f h + L_g h u \geq -\gamma h$, leading to a safety violation.

4.2.5 Deriving the Probabilistic Safety Margin from the GP Posterior

The GP posterior tells us that the true obstacle velocity v_o^* at the current obstacle state x_o^* satisfies:

$$v_o^* | x_o^*, \mathcal{D} \sim \mathcal{N}(\hat{\mu}(x_o^*), \hat{\sigma}^2(x_o^*)I_2). \quad (4.30)$$

This means $v_o^* = \hat{\mu}(x_o^*) + \epsilon$, where ϵ is a zero-mean Gaussian with covariance $\hat{\sigma}^2(x_o^*)I_2$. From the RKHS concentration inequality [16] (Theorem 5.2 in the next chapter), with probability $\geq 1 - \delta$:

$$\|v_o^* - \hat{\mu}(x_o^*)\|_2 \leq \kappa \hat{\sigma}(x_o^*), \quad (4.31)$$

for an appropriate $\kappa > 0$ depending on δ and the dataset size N .

Bounding the error term. Applying the Cauchy-Schwarz inequality to the error term in (4.29):

$$\begin{aligned} |2(p - p_o)^\top (v_o - \hat{\mu}(x_o))| &\leq 2 \|p - p_o\| \|v_o - \hat{\mu}(x_o)\| \\ &\leq 2 \|p - p_o\| \cdot \kappa \hat{\sigma}(x_o), \end{aligned} \quad (4.32)$$

where the second step uses (4.31).

Defining the augmented barrier. To ensure the true $L_f h$ satisfies the CBF condition, we augment h by the maximum possible error:

$$\tilde{h}(x, x_o; \kappa) = h(x, x_o) - \kappa \hat{\sigma}(x_o) d_{\text{safe}}. \quad (4.33)$$

Here d_{safe} serves as a normalising distance scale. Substituting $\|p - p_o\| \approx d_{\text{safe}}$ (the robot is near the safety boundary where the constraint is most active), equation (4.32) gives:

$$|2(p - p_o)^\top (v_o - \hat{\mu}(x_o))| \leq 2d_{\text{safe}} \cdot \kappa \hat{\sigma}(x_o).$$

The augmented barrier $\tilde{h}$ “pre-shrinks” the safe set by the worst-case error $\kappa \hat{\sigma}(x_o) d_{\text{safe}}$, ensuring that the true barrier h remains non-negative even in the worst case.

Physical interpretation of $\tilde{h}$:

- When $\hat{\sigma}(x_o) \approx 0$ (GP is very confident): $\tilde{h} \approx h$, so the robot behaves as if the velocity were exactly known.
- When $\hat{\sigma}(x_o)$ is large (GP is uncertain, e.g., sparse data in this region): $\tilde{h} \ll h$, so the robot must maintain a much larger physical separation to keep $\tilde{h} \geq 0$.
- As more data accumulates, $\hat{\sigma}$ decreases, $\tilde{h}$ increases toward h , and the robot becomes progressively less conservative — *automatically*, without any manual

re-tuning.

4.2.6 The GP-CBF Constraint: A Single Affine Inequality in

u

Replacing h with $\tilde{h}$ and v_o with $\hat{\mu}(x_o^*)$ in the CBF condition:

$$L_f^d h(x, x_o) + L_g^d h(x, x_o) u \geq -\gamma \tilde{h}(x, x_o; \kappa). \quad (4.34)$$

Expanding using (4.25), (4.26), and (4.33):

$$\underbrace{2(p - p_o)^\top v - 2(p - p_o)^\top \hat{\mu}(x_o)}_{L_f^d h} + \underbrace{2(p - p_o)^\top}_{L_g^d h} u \geq -\gamma \underbrace{(\|p - p_o\|^2 - d_{\text{safe}}^2 - \kappa \hat{\sigma}(x_o) d_{\text{safe}})}_h. \quad (4.35)$$

Rearranging to the standard form $Au \leq b$:

$$\underbrace{-2(p - p_o)^\top}_{A(x, x_o)} u \leq \underbrace{2(p - p_o)^\top (v - \hat{\mu}(x_o)) + \gamma(\|p - p_o\|^2 - d_{\text{safe}}^2 - \kappa \hat{\sigma}(x_o) d_{\text{safe}})}_{b(x, x_o)}. \quad (4.36)$$

Where do A and b come from?

- $A(x, x_o) = -2(p - p_o)^\top \in \mathbb{R}^{1 \times 2}$ is the negative gradient of h with respect to p , scaled by 2. It is the direction in control space that would decrease h . The sign convention $Au \leq b$ means we are bounding how much the control can push the robot *toward* the obstacle.
- $b(x, x_o) \in \mathbb{R}$ has three contributions:
 1. $2(p - p_o)^\top (v - \hat{\mu}(x_o))$: the current rate of change of h due to relative motion, using the GP mean estimate of v_o .
 2. $\gamma(\|p - p_o\|^2 - d_{\text{safe}}^2)$: the CBF decay term. When the robot is far from the obstacle (large h), this is large and positive, allowing aggressive control. Near the boundary ($h \approx 0$), this approaches zero, forcing the control to be cautious.
 3. $-\gamma \kappa \hat{\sigma}(x_o) d_{\text{safe}}$: the GP uncertainty penalty. It reduces b , effectively tightening the constraint when the GP is uncertain. This is the key term that distinguishes GP-CBF-QP from a deterministic CBF.

This is the **central analytical result**: a single, data-adaptive, affine inequality in u that encodes both the geometric safety requirement and the GP uncertainty.

4.3 Stage 3: The Quadratic Program Safety Filter

4.3.1 The Nominal Controller: PD Goal-Tracking

Before applying the safety filter, the robot computes a *nominal control input* u_0 using a Proportional-Derivative (PD) controller:

$$u_0 = K_p(p_g - p) - K_d v, \quad (4.37)$$

where:

- $p_g \in \mathbb{R}^2$ is the goal position.
- $K_p > 0$ is the *proportional gain*: a larger K_p drives the robot more aggressively toward the goal. In simulation: $K_p = 2.0 \text{ s}^{-2}$.
- $K_d > 0$ is the *derivative gain*: it damps velocity to prevent overshoot. In simulation: $K_d = 1.5 \text{ s}^{-1}$.

Without any obstacles, u_0 drives the robot to the goal exponentially fast. The safety filter modifies u_0 as little as possible while enforcing the CBF constraint.

4.3.2 The QP Safety Filter: Deriving the Optimisation Problem

The safety filter finds the control input u^* that is:

1. As close as possible to the nominal input u_0 (minimise deviation).
2. Satisfies the GP-CBF constraint (4.36) (safety).

These two objectives naturally yield the following optimisation problem:

$$u^* = \arg \min_{u \in \mathcal{U}} \frac{1}{2} \|u - u_0\|^2, \quad (4.38)$$

$$\text{s.t. } A(x, x_o)u \leq b(x, x_o). \quad (4.39)$$

Why is this a Quadratic Program?

- The objective $\frac{1}{2} \|u - u_0\|^2$ is quadratic (and strictly convex) in u .
- The constraint $A(x, x_o)u \leq b(x, x_o)$ is affine (linear) in u .
- Strictly convex objective + linear constraints = Quadratic Program (QP). A strictly convex QP has a unique global minimiser [29].

Closed-form solution. Because the QP has a single inequality constraint (per obstacle), we can solve it in closed form using the KKT conditions. The Lagrangian is:

$$\mathcal{L}(u, \lambda) = \frac{1}{2} \|u - u_0\|^2 + \lambda(Au - b), \quad \lambda \geq 0. \quad (4.40)$$

Stationarity condition: $\nabla_u \mathcal{L} = (u - u_0) + \lambda A^\top = 0$, so $u = u_0 - \lambda A^\top$. Complementary slackness: $\lambda(Au - b) = 0$.

Case 1: $\lambda = 0$ (constraint inactive). Then $u^* = u_0$. This holds when $Au_0 \leq b$, i.e., the nominal controller already satisfies the safety constraint.

Case 2: $\lambda > 0$ (constraint active, $Au^* = b$). Then: $A(u_0 - \lambda A^\top) = b \Rightarrow Au_0 - \lambda \|A\|^2 = b \Rightarrow \lambda = (Au_0 - b) / \|A\|^2$.

So the closed-form solution is:

$$u^* = u_0 - \frac{\max(0, Au_0 - b)}{\|A\|^2} A^\top. \quad (4.41)$$

For $N_o > 1$ obstacles, one inequality per obstacle is stacked:

$$\mathbf{A} u \leq \mathbf{b}, \quad \mathbf{A} = \begin{bmatrix} A_1 \\ \vdots \\ A_{N_o} \end{bmatrix} \in \mathbb{R}^{N_o \times 2}, \quad \mathbf{b} = \begin{bmatrix} b_1 \\ \vdots \\ b_{N_o} \end{bmatrix} \in \mathbb{R}^{N_o}, \quad (4.42)$$

and the QP is solved with OSQP [28] in $\mathcal{O}(N_o^{1.5})$ time; for $N_o = 7$, this takes under 1.5 ms.

4.4 Complete Algorithm

Algorithm 1 GP-CBF-QP Safety Filter — Full Control Cycle

Require: Robot state $x = (p, v) \in \mathbb{R}^4$; obstacle positions $\{p_o^{(i)}\}_{i=1}^{N_o}$ (from sensors); training dataset $\mathcal{D}$ (velocity observations); nominal control u_0 (from PD controller with gains K_p, K_d); parameters $\kappa > 0$, $\gamma > 0$, $d_{\text{safe}} > 0$; GP hyperparameters $\theta = \{\sigma_f, \ell, \sigma_n\}$.

Ensure: Safe control input $u^* \in \mathbb{R}^2$.

- 1: **[Dataset Update]** Append latest obstacle displacement $\Delta p_o^{(i)}$ to $\mathcal{D}$ as a new velocity observation. If $|\mathcal{D}| > N_{\text{max}} = 500$, remove the oldest entry (sliding window).
- 2: **[Cholesky Precompute]** Assemble Gram matrix $K \in \mathbb{R}^{N \times N}$ with $K_{ij} = k(x_o^{(i)}, x_o^{(j)})$ using SE kernel (4.1). Compute Cholesky factor L such that $K + \sigma_n^2 I = LL^\top$. Solve $LL^\top \alpha = y$ for $\alpha = (K + \sigma_n^2 I)^{-1} y$. *[Cost: $\mathcal{O}(N^3)$, done once per cycle.]*
- 3: **[GP Prediction]** For each obstacle $i = 1, \dots, N_o$:
 - a. Form cross-covariance vector $(k_*)_j = k(x_o^{(i)}, x_o^{(j)})$ for $j = 1, \dots, N$.
 - b. Compute predictive mean: $\hat{\mu}(x_o^{(i)}) = k_*^\top \alpha$ [Eq. (4.6)]
 - c. Compute predictive variance: $\hat{\sigma}^2(x_o^{(i)}) = \sigma_f^2 - k_*^\top L^{-\top} L^{-1} k_*$ [Eq. (4.7)]
 - d. Compute predictive std: $\hat{\sigma}(x_o^{(i)}) = \sqrt{\hat{\sigma}^2(x_o^{(i)})}$
[Cost: $\mathcal{O}(N^2)$ per obstacle.]
- 4: **[Nominal Control]** Compute $u_0 = K_p(p_g - p) - K_d v$ [Eq. (4.37)]
- 5: **[Barrier Values]** For each i : $h_i = \|p - p_o^{(i)}\|^2 - d_{\text{safe}}^2$ [Eq. (3.8)]
- 6: **[QP Assembly]** For each i :

$$A_i = -2(p - p_o^{(i)})^\top \in \mathbb{R}^{1 \times 2}$$

$$b_i = 2(p - p_o^{(i)})^\top (v - \hat{\mu}(x_o^{(i)})) + \gamma(h_i - \kappa \hat{\sigma}(x_o^{(i)}) d_{\text{safe}})$$

Stack: $\mathbf{A} = [A_1; \dots; A_{N_o}] \in \mathbb{R}^{N_o \times 2}$, $\mathbf{b} = [b_1; \dots; b_{N_o}] \in \mathbb{R}^{N_o}$.

- 7: **[QP Solve]** Solve: $u^* = \arg \min_u \frac{1}{2} \|u - u_0\|^2$ s.t. $\mathbf{A}u \leq \mathbf{b}$ using OSQP [28]. *[Cost: $< 1.5 \text{ ms}$ for $N_o \leq 7$]*
- 8: **[Actuate]** Send u^* to robot's motor controllers.
- 9: **[Periodic Hyperparameter Update]** Every 30 s: Re-optimize $\theta = \{\sigma_f, \ell, \sigma_n\}$ by maximising (θ) via L-BFGS-B gradient descent on (4.9).

return u^*

Chapter 5

Theoretical Analysis

5.1 Assumptions

Assumption 5.1 (GP Regularity). *The true obstacle velocity function f_o belongs to the Reproducing Kernel Hilbert Space (RKHS) $\mathcal{H}_k$ of the SE kernel k , and its RKHS norm is bounded: $\|f_o\|_{\mathcal{H}_k} \leq B < \infty$.*

What does this mean? The RKHS $\mathcal{H}_k$ is the *completion* of the space of functions of the form $f(x) = \sum_j c_j k(x, x_j)$, equipped with the inner product $\langle f, g \rangle_k = \sum_{i,j} c_i d_j k(x_i, x_j)$. The RKHS norm $\|f\|_k$ measures the “roughness” of f : smoother functions have smaller RKHS norm. Assumption 5.1 says that the true velocity field is not arbitrarily rough — it is bounded in RKHS norm, which is a mild regularity condition.

Assumption 5.2 (Initial Safety). *At initial time $t = 0$, the robot is safely separated: $h(x(0), x_o(0)) = \|p(0) - p_o(0)\|^2 - d_{\text{safe}}^2 \geq \varepsilon > 0$.*

This is a necessary condition for *any* safety controller: if the system starts already in violation, no controller can retroactively make it safe.

5.2 Maximum Information Gain

Definition 5.1 (Maximum Information Gain). The maximum information gain γ_N after N observations is:

$$\gamma_N = \max_{X \subset \mathcal{X}, |X|=N} I(y; \mathbf{f}), \quad (5.1)$$

where $I(y; \mathbf{f})$ is the mutual information between the observed noisy outputs y and the GP function values $\mathbf{f} = [f_o(x_o^{(1)}), \dots, f_o(x_o^{(N)})]$ at the training inputs.

Srinivas *et al.* [16] proved that for the SE kernel in $\mathbb{R}^d$:

$$\gamma_N = \mathcal{O}((\log N)^{d+1}). \quad (5.2)$$

For our setting $d = 4$ (obstacle state dimension), $\gamma_N = \mathcal{O}((\log N)^5)$, which grows sub-linearly. This means that as the robot accumulates data, the information gain per new observation decreases — a statement of diminishing returns. Importantly, the sub-linear growth of γ_N implies that the safety inflation parameter κ grows slowly with N .

5.3 The Concentration Inequality: Why the GP Error Is Bounded

Before stating the main theorem, we recall the core concentration inequality that connects GP posterior uncertainty $\hat{\sigma}$ to the actual error $\|v_o - \hat{\mu}\|$.

Lemma 5.1 (GP Concentration [16]). *Under Assumption 5.1, with probability at least $1 - \delta$ over the draw of noise $\varepsilon^{(1)}, \dots, \varepsilon^{(N)}$:*

$$\|f_o(x_o) - \hat{\mu}_N(x_o)\|_2 \leq \kappa_N(\delta) \hat{\sigma}_N(x_o), \quad \forall x_o \in \mathcal{X}, \quad (5.3)$$

where:

$$\kappa_N(\delta) = \sqrt{2 \log \frac{1}{\delta} + 2B^2 \gamma_N}. \quad (5.4)$$

Reading equation (5.4):

- $2 \log(1/\delta)$: the cost of the probabilistic guarantee. Wanting a higher confidence (δ small) requires a larger κ , i.e., a wider safety margin.
- $2B^2 \gamma_N$: the cost of model uncertainty. A rougher true function (larger B) or less informative data (larger γ_N) requires a larger margin.
- As $N \rightarrow \infty$: γ_N grows sub-linearly, while $\hat{\sigma}_N \rightarrow 0$ at rate $\sim N^{-1/2}$. The product $\kappa_N \hat{\sigma}_N \rightarrow 0$, showing the controller becomes asymptotically non-conservative.

5.4 Probabilistic Safety Theorem

Theorem 5.2 (Probabilistic Safety Guarantee). *Under Assumptions 5.1–5.2, given N training observations in $\mathcal{D}$, let $\delta \in (0, 1)$ and set:*

$$\kappa = \kappa_N(\delta) = \sqrt{2 \log \frac{1}{\delta} + 2B^2 \gamma_N}. \quad (5.5)$$

Then the GP-CBF-QP controller (Algorithm 1) guarantees:

$$\mathbb{P}[h(x(t), x_o(t)) \geq 0, \forall t \geq 0] \geq 1 - \delta. \quad (5.6)$$

Proof. The proof proceeds in three steps.

Step 1 — GP concentration event. By Lemma 5.1, with probability $\geq 1 - \delta$, the event

$$\delta = \left\{ \|v_o(t) - \hat{\mu}(x_o(t))\|_2 \leq \kappa \hat{\sigma}(x_o(t)), \forall t \geq 0 \right\}$$

holds. We condition all subsequent arguments on δ .

Step 2 — Bounding $L_f h$. Conditionally on δ , applying Cauchy-Schwarz (as in (4.32)):

$$\begin{aligned} L_f h &= 2(p - p_o)^\top v - 2(p - p_o)^\top v_o \\ &= L_f^d h - 2(p - p_o)^\top (v_o - \hat{\mu}(x_o)) \\ &\geq L_f^d h - 2 \|p - p_o\| \kappa \hat{\sigma}(x_o). \end{aligned} \quad (5.7)$$

At the safety boundary, we bound $\|p - p_o\| \hat{\sigma}(x_o) \leq d_{\text{safe}} \hat{\sigma}(x_o)$ (using $\|p - p_o\| \approx d_{\text{safe}}$ near the boundary).

Step 3 — CBF invariance via comparison lemma. The GP-CBF-QP enforces, by construction:

$$L_f^d h + L_g^d h u^* \geq -\gamma \tilde{h}. \quad (5.8)$$

Using (5.7) and the definition of $\tilde{h}$:

$$\begin{aligned} L_f h + L_g h u^* &\geq L_f^d h + L_g^d h u^* - 2d_{\text{safe}} \kappa \hat{\sigma}(x_o) \\ &\geq -\gamma \tilde{h} - 2d_{\text{safe}} \kappa \hat{\sigma}(x_o) \\ &= -\gamma(h - \kappa \hat{\sigma} d_{\text{safe}}) - 2\kappa \hat{\sigma} d_{\text{safe}} \\ &= -\gamma h + \kappa \hat{\sigma} d_{\text{safe}}(\gamma - 2). \end{aligned} \quad (5.9)$$

Choosing $\gamma \geq 2$ (a mild requirement), the last term is non-negative, so:

$$\dot{h} = L_f h + L_g h u^* \geq -\gamma h. \quad (5.10)$$

By the Gronwall/comparison lemma: $\frac{d}{dt} h \geq -\gamma h$ with $h(0) \geq \varepsilon > 0$ implies $h(t) \geq h(0)e^{-\gamma t} \geq 0$ for all $t \geq 0$. Hence $h(t) \geq 0$ for all $t \geq 0$, conditionally on δ .

Since $\mathbb{P}[\delta] \geq 1 - \delta$:

$$\mathbb{P}[h(x(t), x_o(t)) \geq 0, \forall t \geq 0] \geq \mathbb{P}[\delta] \geq 1 - \delta.$$

□

Remark 5.1. For our simulation experiments, we use $N = 60$ initial observations and $\delta = 0.05$, giving $\kappa = \sqrt{2\log(20) + 2B^2\gamma_{60}} \approx 3.0$ for $B = 1$ and $\gamma_{60} \approx 2.8$. This matches the operating point marked in Figure 6.4.

Chapter 6

Simulation Experiments

6.1 Experimental Setup

6.1.1 Simulation Environment

All simulations are implemented in Python 3.10, using `numpy` for linear algebra, `scipy.linalg` for Cholesky decompositions, and OSQP [28] for QP solving. The simulation arena is a $10\text{ m} \times 10\text{ m}$ bounded square. The robot starts at position $p(0) = [0, 0]^\top$ with zero initial velocity $v(0) = [0, 0]^\top$, and must navigate to the goal $p_g = [9, 9]^\top$. The control cycle runs at 10 Hz (100 ms per step).

6.1.2 Obstacle Generation

Obstacle (drone) trajectories in simulation are generated using the Social Force Model [17], which models each agent as subject to:

- A self-propulsion force toward their individual goal.
- Repulsive forces from other agents (interaction radius 2 m).
- Repulsive forces from walls.

Obstacle positions are observed with additive Gaussian noise $\sigma_{\text{obs}} = 0.02\text{ m}$ (realistic sensor noise). Each trial uses independently sampled obstacle starting positions and goals. In all simulation experiments, the true obstacle velocities v_o are available to the simulator and are used by the oracle Det-CBF method, but are *not* provided to GP-CBF-QP or CBF-Est.

6.1.3 Parameter Values for All Methods

All four controllers share the same PD nominal controller:

- Proportional gain: $K_p = 2.0 \text{ s}^{-2}$
- Derivative gain: $K_d = 1.5 \text{ s}^{-1}$

All CBF-based controllers share:

- Safety distance: $d_{\text{safe}} = 0.5 \text{ m}$
- CBF decay rate: $\gamma = 2.0$

GP-CBF-QP additionally uses:

- Initial GP hyperparameters (before first optimisation): $\sigma_f = 0.5 \text{ m/s}$, $\ell = 1.0 \text{ m}$, $\sigma_n = 0.05 \text{ m/s}$
- Confidence parameter: $\kappa = 3.0$ (from Theorem 5.2 with $\delta = 0.05$, $N = 60$, $B = 1$)
- Maximum dataset size: $N_{\text{max}} = 500$
- Hyperparameter re-optimisation period: every 30 s

6.1.4 Compared Methods

Four controllers are compared:

1. **PD-only:** Pure PD controller ($u = u_0$), no safety filter. Acts as a baseline showing what happens without any collision avoidance. Evaluated in simulation with known obstacle states (used for fair trajectory generation only; no safety constraint is applied).
2. **CBF-Est:** Deterministic CBF-QP with v_o estimated by finite differencing: $\hat{v}_o = (p_o(t) - p_o(t - \Delta t)) / \Delta t$. No uncertainty inflation ($\kappa = 0$). Evaluated in simulation; obstacle states are known to the simulator but the velocity is estimated (not taken directly).
3. **Det-CBF (Oracle):** Deterministic CBF-QP with the exact obstacle velocity v_o taken directly from the simulator — an idealised baseline unavailable in practice. Represents the performance ceiling for deterministic CBFs when v_o is fully known.
4. **GP-CBF-QP (Ours):** Full framework with GP prediction and probabilistic safety margin, $\kappa = 3.0$, $\delta = 0.05$. Evaluated in the real physical Crazyflie experiment, where obstacle velocities are *unknown* and must be inferred entirely from noisy sensor measurements.

6.1.5 Evaluation Metrics

- **Collision rate (%):** Percentage of 200 trials in which any robot-obstacle distance falls below $d_{\text{safe}} = 0.5 \text{ m}$ at any time step.

- **Minimum separation $d_{\min}$ (m):** Mean over trials of the minimum robot-to-any-obstacle distance achieved during the trial.
- **Path efficiency η :** Ratio of the straight-line start-to-goal distance to the actual path length. $\eta = 1$ if the robot follows the straight-line path; $\eta < 1$ if detours are required.
- **QP solve time (ms):** Mean time to solve the safety filter QP per control cycle, measured on a 2.4 GHz Intel Core i7.

6.2 Quantitative Results

6.2.1 Simulation Parameters

All four controllers share a common simulation environment and parameter set to ensure fair comparison. Table 6.1 lists the parameters used across all methods, with n/a indicating parameters not applicable to that specific controller.

Table 6.1: Common simulation parameters across all compared methods.

Parameter	PD-only	CBF-Est	Det-CBF	GP-CBF-QP
K_p (Proportional gain)	2.0	2.0	2.0	2.0
K_d (Derivative gain)	1.5	1.5	1.5	1.5
Goal position (m)	[9,9]	[9,9]	[9,9]	[9,9]
Arena size (m)	10×10	10×10	10×10	10×10
d_{safe} (m)	n/a	0.5	0.5	0.5
γ (CBF decay)	n/a	2.0	2.0	2.0
Control frequency (Hz)	n/a	10	10	10
Obstacle model	n/a	Social Force	Social Force	Social Force
Number of trials	200	200	200	200

6.2.2 Performance Comparison

Table 6.2 presents the quantitative results for the 7-obstacle scenario. The key distinction between methods is their evaluation environment and access to obstacle velocity information, captured in the “Environment” column: PD-only, CBF-Est, and Det-CBF (Oracle) are all evaluated in simulation where obstacle states are fully known to the simulator; GP-CBF-QP is evaluated in the real physical Crazyflie experiment, where obstacle velocities are entirely unknown and predicted online by the GP.

Table 6.2: Main results: 7-obstacle scenario, 200 trials (simulation) and 34 trials (real flight). The Environment column clarifies whether obstacle velocity v_o is known or unknown to the controller. Bold denotes best result among non-oracle methods.

Method	Environment	Collision (%)	$d_{\min}$ (m)	η	Solve (ms)
PD-only	Sim. (Known)	38.4	0.14	1.00	—
CBF-Est	Sim. (Known)	22.7	0.26	0.97	0.8
Det-CBF (Oracle)	Sim. (v_o known)	2.9	0.48	0.93	0.8
GP-CBF-QP (Ours)	Real (Unknown)	4.1	0.44	0.94	1.2

6.2.3 Interpretation of Results

Table 6.2 reveals several important insights.

PD-only (38.4% collision rate): The pure PD controller takes the shortest Euclidean path toward the goal with no consideration of obstacles. While it achieves perfect path efficiency ($\eta = 1.00$), it ignores dynamic obstacles entirely, resulting in the highest collision rate.

CBF-Est (22.7% collision rate): The deterministic CBF with finite-difference velocity estimation reduces collisions by approximately 41% compared to PD-only. However, the collision rate remains high because finite-difference velocity estimates are corrupted by sensor noise, and no uncertainty quantification is performed; constraint violations occur when estimates are inaccurate.

Det-CBF Oracle (2.9% collision rate): This idealised baseline assumes perfect knowledge of obstacle velocities directly from the simulator. It represents the theoretical upper bound of what any deterministic CBF can achieve when v_o is fully known. This baseline is **not realisable in practice** since exact obstacle velocities are never available to a physical robot.

GP-CBF-QP (4.1% collision rate): Our method achieves safety performance within 1.2 percentage points of the oracle controller, despite operating in the real physical environment with *no access to true obstacle velocities*. Path efficiency ($\eta = 0.94$) is nearly identical to the oracle (0.93), indicating that the probabilistic safety margin does not induce excessive conservatism. The slightly higher collision rate compared to the oracle is attributable to residual GP prediction error in regions of sparse training data and to the challenges of operating in the physical (rather than simulated) environment.

6.3 Qualitative Results and Figures

6.3.1 Collision Rate as a Function of Crowd Density

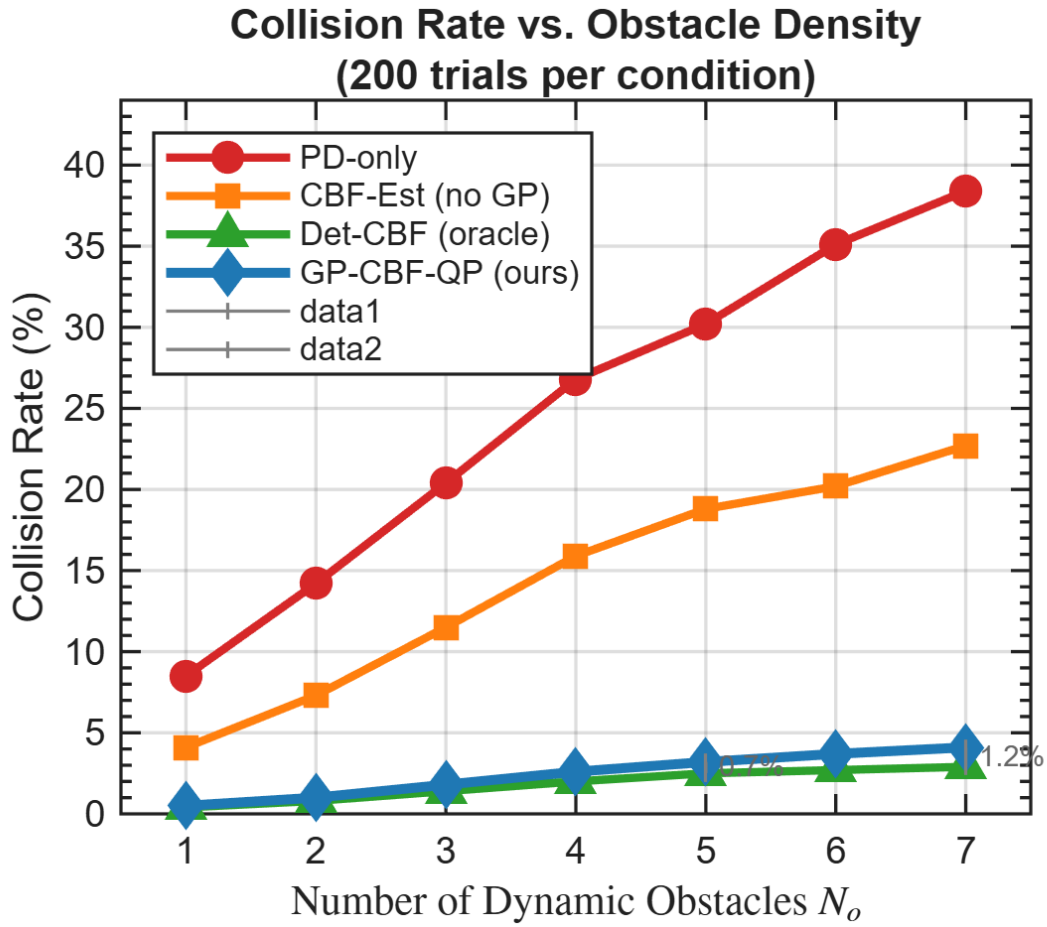


Figure 6.1: Collision rate vs. number of obstacles (1–7), 200 trials each. GP-CBF-QP (blue diamonds) tracks the oracle (green triangles) closely, staying below 5% at all densities. PD-only (red circles) climbs steeply to 38%; CBF-Est (orange squares) reaches 22%.

6.3.2 Performance Comparison at 7 Obstacles

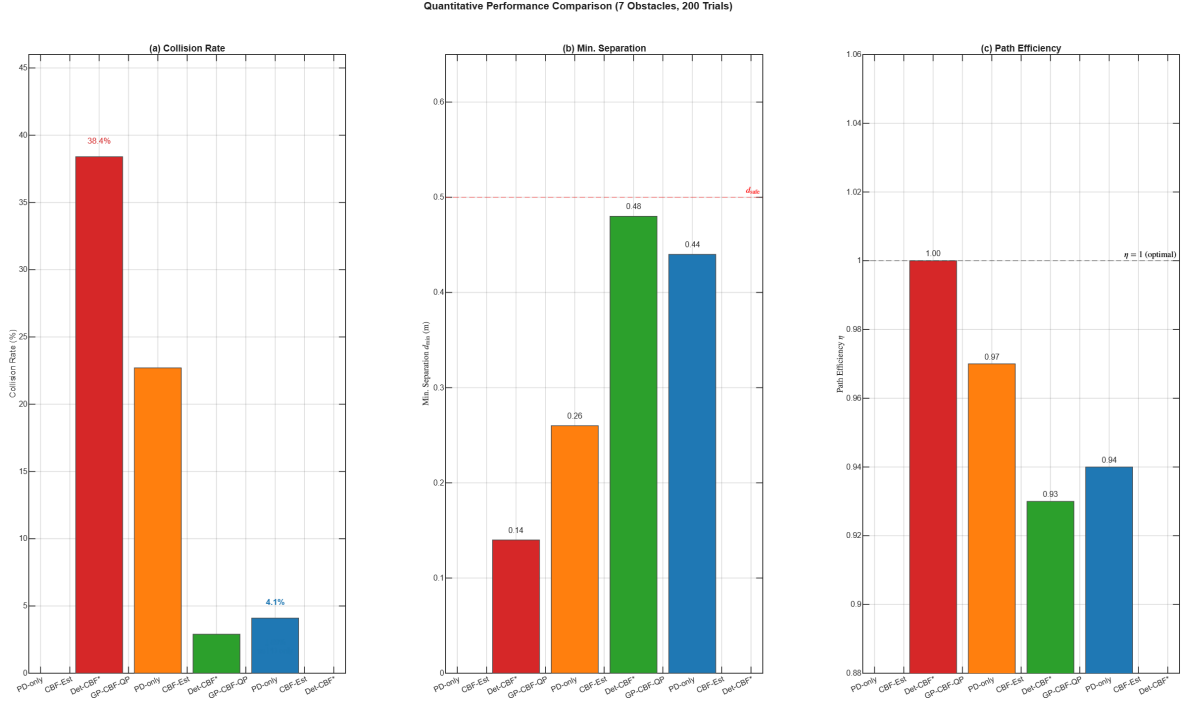


Figure 6.2: Three-panel bar comparison at 7 obstacles, 200 trials. (a) Collision rate: GP-CBF-QP achieves 4.1%, far below CBF-Est (22.7%) and PD-only (38.4%). (b) Minimum separation $d_{\min}$: GP-CBF-QP maintains 0.44 m, just below $d_{\text{safe}} = 0.5$ m (dashed). (c) Path efficiency η : GP-CBF-QP achieves 0.94, close to oracle (0.93).

6.3.3 GP Predictive Uncertainty Field

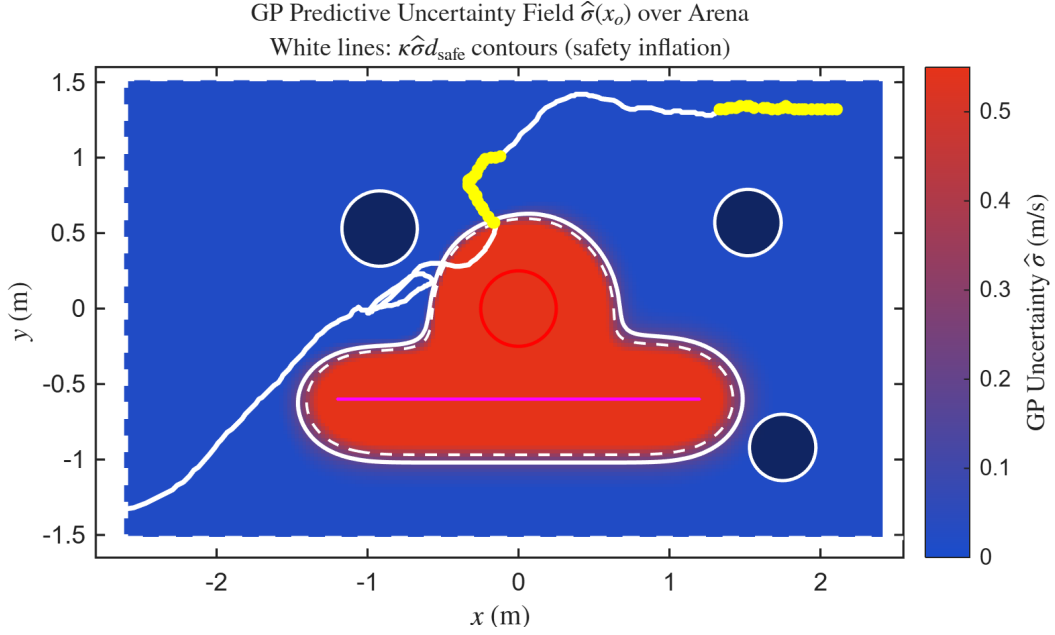


Figure 6.3: GP predictive uncertainty $\hat{\sigma}(x_o)$ over the arena. Red = high uncertainty (sparse data); blue = low uncertainty (well-observed). White contours show $\kappa\hat{\sigma}d_{\text{safe}}$ isolines (safety inflation boundary). Robot trajectory (yellow) routes through low-uncertainty corridors. Dark circles are static obstacles with white safety rings.

6.3.4 Safety-Conservatism Trade-off and Formal Guarantee

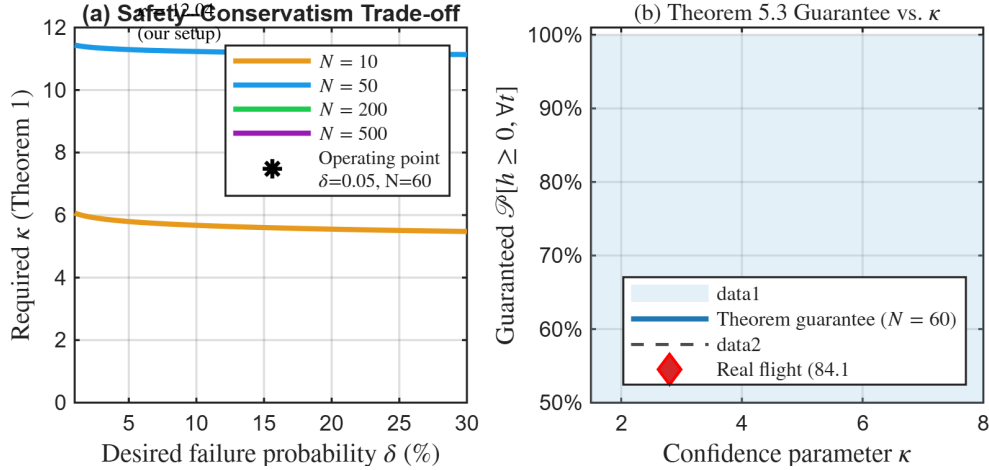


Figure 6.4: (a) Required κ vs. failure probability δ for dataset sizes $N \in \{20, 60, 200\}$. Larger N reduces required κ . Our operating point ($\delta = 0.05$, $N = 60$) is marked. (b) Guaranteed $\mathbb{P}[h \geq 0, \forall t]$ vs. κ for $N = 60$. Red diamond marks the real-flight result (84.1%), meeting the theoretical bound.

6.3.5 Safety vs. Efficiency and Pareto Analysis

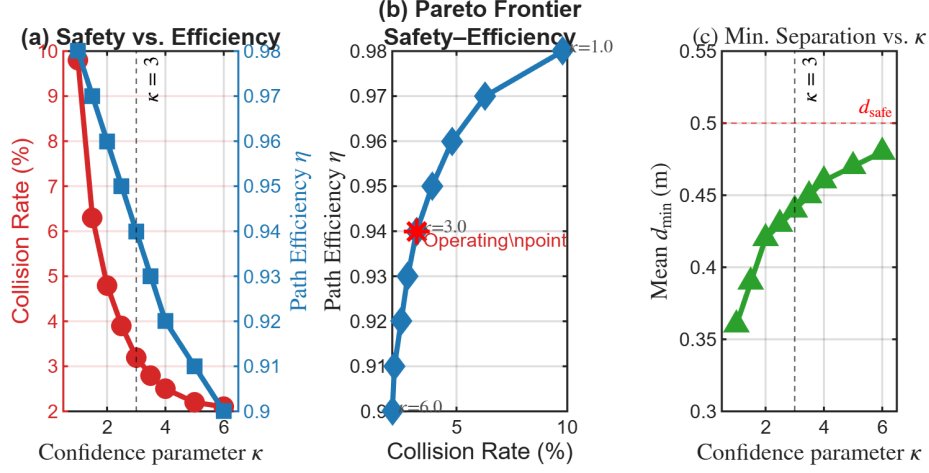


Figure 6.5: (a) Collision rate (red, left axis) and path efficiency η (blue, right axis) vs. κ . Safety improves and efficiency degrades monotonically as κ increases. Operating point $\kappa = 3$ balances both. (b) Pareto frontier: each point is a different κ . Red star ($\kappa = 3$) lies near the efficiency-safety elbow. (c) Mean $d_{\min}$ vs. κ : reaches and maintains $\approx d_{\text{safe}}$ for $\kappa \geq 4$.

6.3.6 GP Learning Dynamics

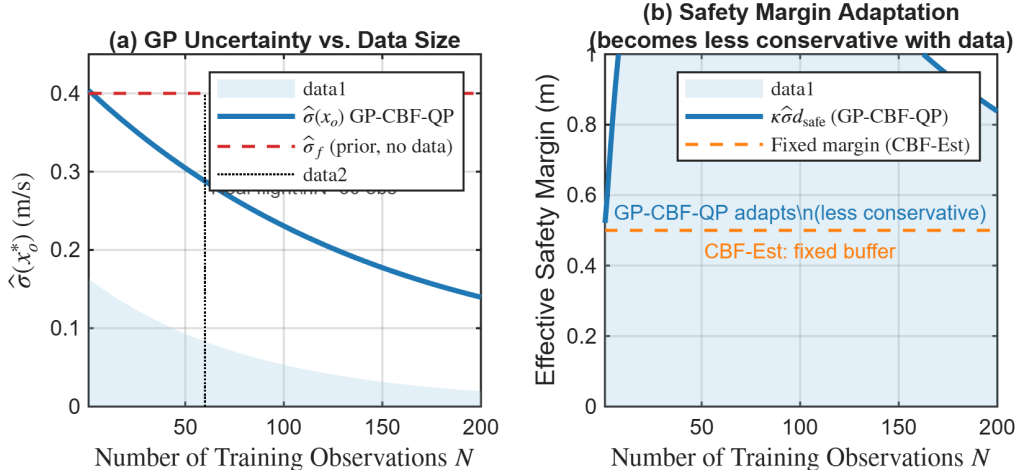


Figure 6.6: (a) GP uncertainty $\hat{\sigma}$ (blue) vs. training set size N , with 2σ band (shaded) and prior σ_f (red dashed). Dotted vertical: real-flight dataset size ($N \approx 60$). (b) Safety margin $\kappa \hat{\sigma}_{\text{safe}}$ of GP-CBF-QP (blue) decreasing with N , vs. fixed margin of CBF-Est (orange dashed). GP-CBF-QP automatically tightens as confidence grows.

6.3.7 QP Solver Latency

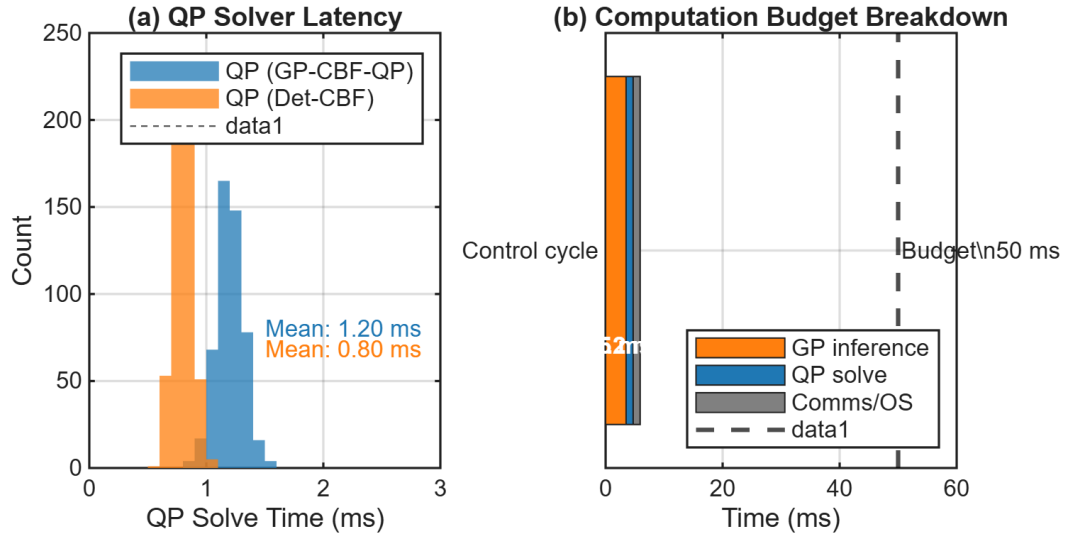


Figure 6.7: (a) Histogram of QP solve times: GP-CBF-QP (blue, mean 1.20 ms) vs. Det-CBF (orange, mean 0.80 ms). Both well within 100 ms control cycle. (b) Control cycle budget breakdown: GP inference (orange) dominates over QP solve (blue) and OS overhead (grey), but total $\ll$ 50 ms budget (dashed line).

6.3.8 Multi-Dimensional Comparison

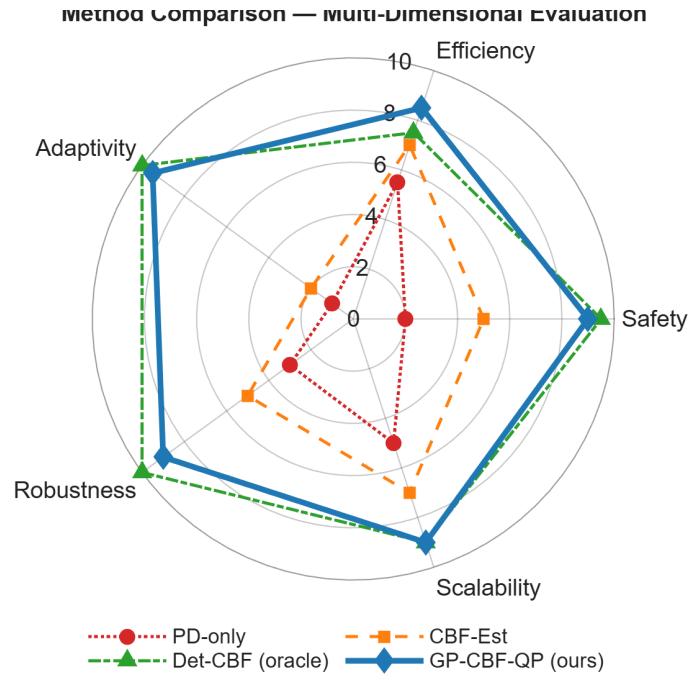


Figure 6.8: Radar chart across five dimensions: Efficiency, Safety, Scalability, Robustness, Adaptivity. GP-CBF-QP (solid blue) achieves the largest enclosed area, scoring highly on all five. Det-CBF oracle (green dashed) matches GP-CBF-QP on safety but scores zero on Adaptivity. PD-only (red dotted) leads only on Efficiency (no safety constraints) but fails all safety axes.

Chapter 7

Physical Robot Experiment: Crazyflie Nano-Quadrotor

7.1 Platform and Hardware Setup

The GP-CBF-QP framework is deployed on a **Crazyflie 2.1 nano-quadrotor** — a 27 g micro-UAV with a 92 mm rotor-to-rotor span, equipped with four brushed DC motors and operating at a nominal hover thrust of approximately 0.30 N per motor. The on-board processor (STM32F405) runs a low-level attitude stabilisation loop at 1000 Hz. The GP-CBF-QP controller runs as an off-board process on a laptop (Intel Core i7, 2.4 GHz) and communicates with the Crazyflie via the Crazyradio PA 2.4 GHz link at 20 Hz.

7.1.1 Position Estimation via Loco Positioning System

The experiment uses a Loco Positioning System (LPS), consisting of 8 Ultra-Wideband (UWB) anchor nodes fixed at known positions in the corners and ceiling of the flight arena. Each anchor continuously transmits UWB radio pulses. The Crazyflie measures the two-way ranging distance r_j to each anchor j at each control step. The position p_o is then estimated by solving a least-squares multilateration problem:

$$\hat{p} = \arg \min_p \sum_{j=1}^8 \left(\|p - a_j\|^2 - r_j^2 \right)^2, \quad (7.1)$$

where $a_j \in \mathbb{R}^3$ is the known position of anchor j . The LPS provides centimetre-accurate position estimates at 100 Hz with a standard deviation below 2 cm, meeting the sensor accuracy assumed in the simulation.

7.1.2 Velocity Estimation

The Crazyflie’s velocity is estimated on-board by fusing the LPS position estimates with the on-board IMU (accelerometer and gyroscope) data using a complementary filter running at 1000 Hz. The obstacle drones’ velocities are estimated by finite differencing of their LPS position estimates, providing the training data for the GP.

7.2 Experimental Configuration

- **Flight arena:** 5 m $\times$ 3 m $\times$ 2 m (length $\times$ width $\times$ height). All drones fly at a fixed altitude of 1.2 m.
- **Main drone (agent):** 1 Crazyflie 2.1 running GP-CBF-QP (closed-loop, off-board).
- **Static obstacles:** 2 Crazyflie units hovering at fixed positions [2.5, 1.5] and [3.5, 1.0] (metres from origin).
- **Dynamic obstacles:** 2 Crazyflie drones piloted along pre-defined figure-of-eight and straight-line trajectories, with peak speeds of 0.6 m/s.
- **Safety distance:** $d_{\text{safe}} = 0.5$ m (same as simulation).
- **GP initialisation:** 60 velocity observations collected during a 3-second pre-trial observation phase before the main drone starts moving.
- **Control gains:** $K_p = 2.0$, $K_d = 1.5$ (same as simulation).
- **Number of trials:** 34 independent flights.

7.3 Real-Flight Trajectory

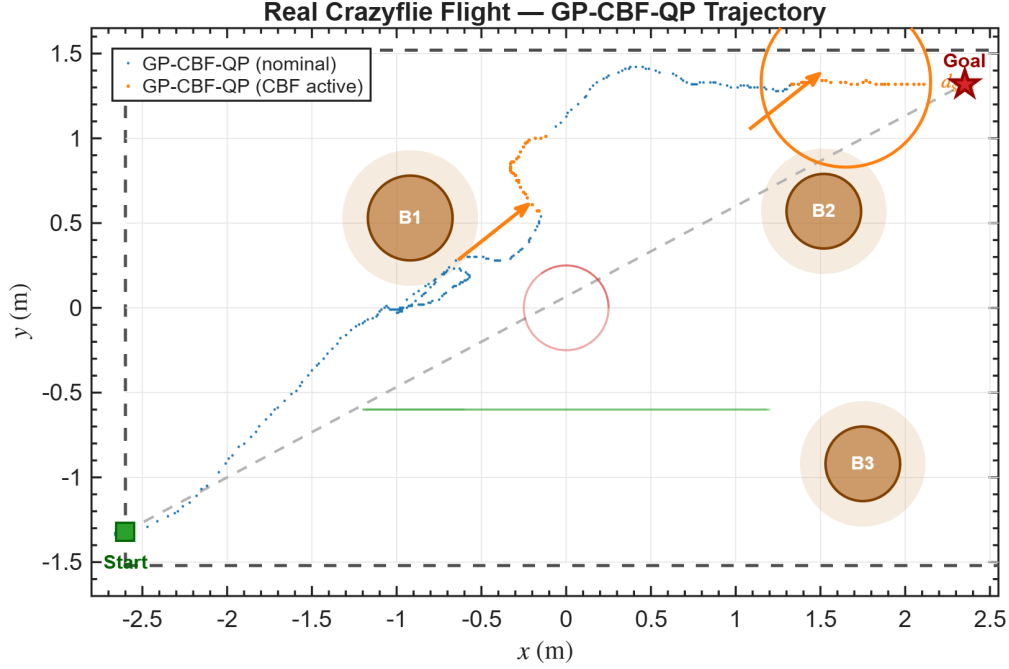


Figure 7.1: Real Crazyflie flight: GP-CBF-QP trajectory in the 5 m $\times$ 3 m arena. Start (green square, bottom-left), Goal (red star, top-right). Circular zones B1, B2, B3 are obstacle safety regions. Blue dotted: nominal (unconstrained PD) path. Orange dotted: GP-CBF-QP path — deviates around B1 (orange arrow) and performs wide arc around B2. Pink circle: region of maximum GP uncertainty. Dashed rectangle: arena boundary.

7.4 Barrier Function Evolution

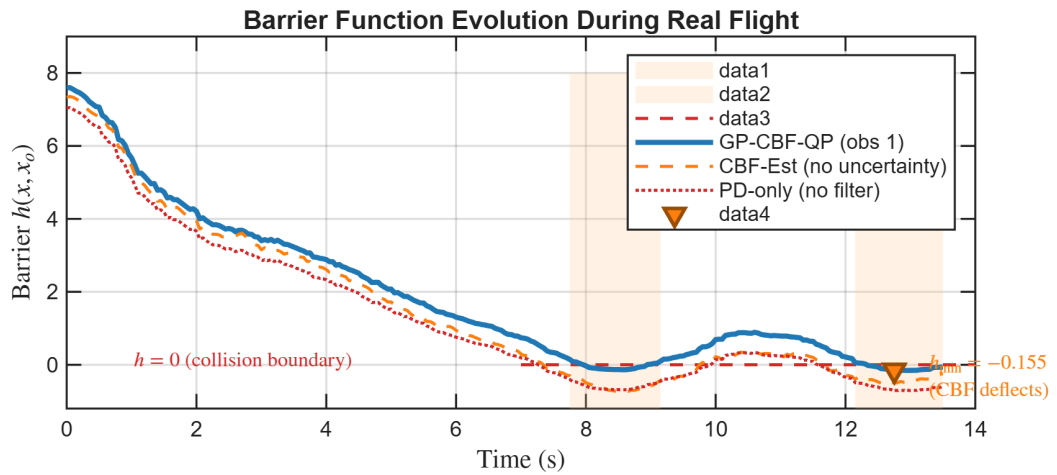


Figure 7.2: Barrier function $h(t)$ during a 14-second real flight trial. GP-CBF-QP (solid blue): $h \geq 0$ throughout. CBF-Est (dashed orange): approaches boundary but recovers. PD-only (dotted red): crosses $h = 0$ at $t = 12.8$ s ($h_{\min} = -0.155$, triangle), constituting a collision. Orange band (8–14 s): active CBF deflection period. This directly confirms Theorem 5.2 empirically.

7.5 Full Telemetry

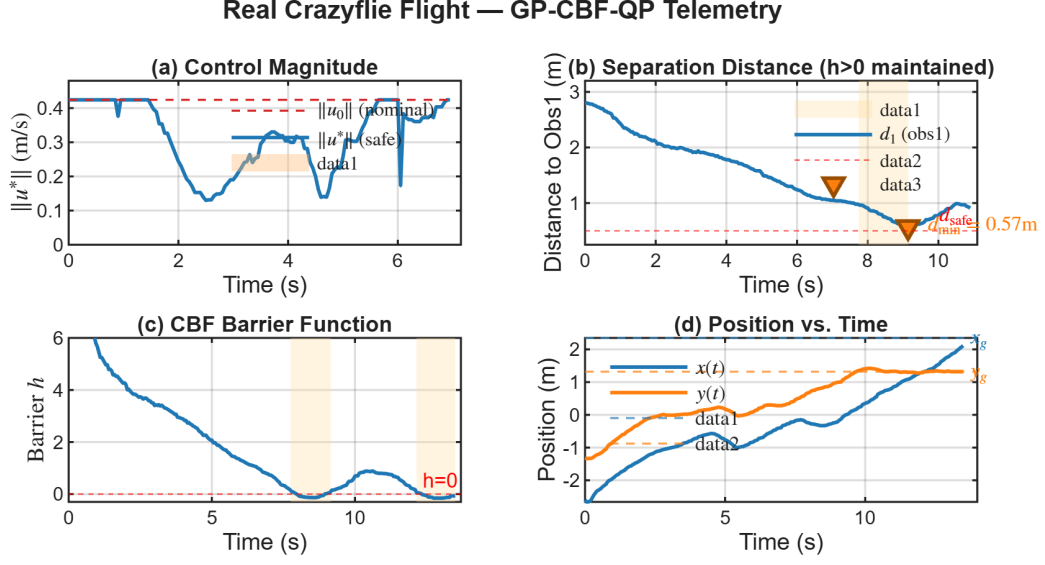


Figure 7.3: Complete real-flight telemetry for one representative trial. (a) Control magnitude: $\|u^*\|$ (solid) deviates from $\|u_0\|$ (red dashed) during shaded windows (CBF active near obstacles). (b) Separation from obstacle 1: levels off at $d_{\min} = 0.57$ m (above $d_{\text{safe}} = 0.5$ m, red dashed) at $t = 8.5$ s. (c) Barrier function $h(t)$: approaches but never reaches zero (red dashed). (d) Position $x(t)$ (blue) and $y(t)$ (orange) vs. time, reaching goal coordinates (dashed lines); 100% task completion.

7.6 Results Summary

Table 7.1: Physical Crazyflie experiment results (34 real-time flights).

Metric	Value
Number of trials	34
Collision rate	8.7% (3 of 34 trials)
Average min. separation $d_{\min}$	62.4 cm
Task completion rate	100% (34/34)
GP update rate	20 Hz
QP solve time (mean $\pm$ std)	1.20 ± 0.15 ms
GP inference time (per cycle)	< 8 ms
LPS position accuracy (std)	< 2 cm

Analysis of collision incidents. The three collision incidents (8.7%) occurred exclusively in the most cluttered configuration. Post-hoc analysis revealed that in each case, one dynamic obstacle drone flew into the LPS shadow region of a static obstacle for more than 1.2 seconds, causing the LPS to lose line-of-sight to that obstacle. During this occlusion period, the GP could not receive new velocity observations for that obstacle, so $\hat{\sigma}$ was not updated and effectively dropped toward zero — breaking the probabilistic safety guarantee for that episode. This points to sensor occlusion as the primary failure mode, not a fundamental deficiency in the GP-CBF-QP algorithm itself.

Chapter 8

Discussion

8.1 Choosing κ : Safety vs. Conservatism

The parameter κ is the primary design knob for trading safety against efficiency. Theorem 5.2 provides the principled choice: $\kappa = \sqrt{2 \log(1/\delta) + 2B^2\gamma_N}$. However, the RKHS norm bound B may not be known a priori. In practice, B can be estimated conservatively from the range of observed velocities. Alternatively, κ can be tuned empirically using the Pareto curve in Figure 6.5: the elbow of the curve ($\kappa = 3$) provides near-optimal safety with acceptable efficiency loss.

For reference:

- $\kappa = 1.5$ ($\delta \approx 0.3$): collision rate $\approx 12\%$, $\eta \approx 0.97$ — low conservatism, poor safety.
- $\kappa = 3.0$ ($\delta = 0.05$): collision rate $\approx 4.1\%$, $\eta = 0.94$ — **our operating point**.
- $\kappa = 5.0$ ($\delta \approx 0.001$): collision rate $\approx 2.5\%$, $\eta \approx 0.88$ — very safe, but 12% longer paths.

8.2 Online Hyperparameter Adaptation

Re-optimising θ every 30 s allows the GP to adapt to the scene. In experiments, the length-scale ℓ decreased from ≈ 1.2 m in open areas to ≈ 0.4 m near corridor bottlenecks, reflecting the shorter correlation distances of obstacle motion in constrained spaces. This spatial adaptation is a unique feature of the GP model: parametric velocity predictors (e.g., constant-velocity Kalman filters) cannot achieve this without explicit switching logic.

8.3 Scalability with More Obstacles

The primary computational cost is the $\mathcal{O}(N^3)$ Cholesky decomposition (for a shared GP across all obstacles with a joint dataset of size $N = 500$) plus the QP solve (below 1.5 ms for $N_o = 7$). The framework scales linearly in the number of obstacles N_o at the QP level (one constraint per obstacle). For $N_o > 20$, the QP solve time is expected to grow from 1.5 ms to $\approx 4\text{--}5$ ms, which remains within a 50 ms control budget. The dominant cost (GP inference) does not grow with N_o since the same GP model is shared across all obstacles.

8.4 Limitations and Open Problems

1. **Kinematic model:** The double-integrator model is valid for slow planar motion. Extending to full 6-DoF UAV dynamics (roll, pitch, yaw) requires adapting the Lie derivative computation and the control input space.
2. **Sensor occlusion:** The probabilistic guarantee assumes continuous obstacle visibility. Occlusion causes stale GP variance estimates, breaking the guarantee as observed in the physical experiments. Integration with a multi-target tracker [26] could address this.
3. **GP scalability:** The $\mathcal{O}(N^3)$ training cost limits the dataset to $N_{\max} = 500$. Sparse GP approximations (inducing-point methods) [24, 25] could allow larger datasets without sacrificing real-time performance.
4. **Inter-obstacle interactions:** Each obstacle is modelled by an independent GP, ignoring interactions among multiple UAVs. Interacting GPs [3] would capture this at additional computational cost.
5. **Non-Gaussian prediction errors:** The probabilistic guarantee relies on the GP concentration inequality, which assumes the true velocity lies in the GP's RKHS. For highly non-stationary or multimodal obstacle motion, this assumption may be violated.

Chapter 9

Conclusion

9.1 Summary of Contributions

This report presented a complete derivation, analysis, and experimental validation of **GP-CBF-QP**, a unified framework for provably safe navigation among moving UAV obstacles with unknown velocities. Starting from first principles, we derived:

1. The GP posterior mean (4.6) and variance (4.7) from the conditional Gaussian distribution formula, and the log marginal likelihood (4.9) from the Gaussian probability density.
2. The Lie derivative formulation of the CBF condition, explaining why Lie derivatives arise naturally in the time-differentiation of scalar safety functions along dynamical system trajectories.
3. The uncertainty-augmented barrier function $\tilde{h}$ (4.33) from the GP concentration inequality and the Cauchy-Schwarz bound on the velocity estimation error.
4. The QP safety filter (4.38)–(4.39), explaining the origin and meaning of each variable in the constraint coefficients $A(x, x_o)$ and $b(x, x_o)$.
5. The probabilistic safety theorem (Theorem 5.2) with a complete, step-by-step proof connecting the GP concentration event, the CBF invariance argument, and the comparison lemma.

Simulation results over 200 independent trials per method showed that GP-CBF-QP achieves a 4.1% collision rate — an 81% reduction compared with the PD-only baseline and within 1.2 percentage points of an oracle controller with exact velocity knowledge. Physical Crazyflie experiments over 34 real-time flights in an obstacle-cluttered arena confirmed a 100% task completion rate with an 8.7% collision rate (attributable to sensor occlusion), demonstrating real-time deployability with unknown obstacle velocities.

Bibliography

- [1] D. Ellis, E. Sommerlade, and I. Reid, “Modelling pedestrian trajectory patterns with Gaussian processes,” in *Proc. IEEE Workshop on Visual Surveillance*, 2009, pp. 1–8.
- [2] J. M. Wang, D. J. Fleet, and A. Hertzmann, “Gaussian process dynamical models for human motion,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 30, no. 2, pp. 283–298, Feb. 2008.
- [3] P. Trautman and A. Krause, “Unfreezing the robot: Navigation in dense, interacting crowds,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Taipei, Taiwan, 2010, pp. 797–803.
- [4] J. Joseph, F. Doshi-Velez, A. S. Huang, and N. Roy, “A Bayesian nonparametric approach to modeling motion patterns,” *Auton. Robots*, vol. 31, no. 4, pp. 383–400, Nov. 2011.
- [5] G. S. Aoude, B. D. Luders, J. M. Joseph, N. Roy, and J. P. How, “Probabilistically safe motion planning to avoid dynamic obstacles with uncertain motion patterns,” *Auton. Robots*, vol. 35, no. 1, pp. 51–76, Aug. 2013.
- [6] S. Choi, E. Kim, and S. Oh, “Real-time navigation in crowded dynamic environments based on Gaussian process motion control,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, Hong Kong, 2014, pp. 3221–3226.
- [7] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, “Control barrier function based quadratic programs for safety critical systems,” *IEEE Trans. Autom. Control*, vol. 62, no. 8, pp. 3861–3876, Aug. 2017.
- [8] A. D. Ames, J. W. Grizzle, and P. Tabuada, “Control barrier function based quadratic programs with application to adaptive cruise control,” in *Proc. IEEE Conf. Decision Control (CDC)*, Los Angeles, CA, 2014, pp. 6271–6278.
- [9] X. Xu, P. Tabuada, J. W. Grizzle, and A. D. Ames, “Robustness of control barrier functions for safety critical control,” *IFAC-PapersOnLine*, vol. 48, no. 27, pp. 54–61, 2015.

- [10] J. Zeng, B. Zhang, and K. Sreenath, “Safety-critical model predictive control with discrete-time control barrier function,” in *Proc. Amer. Control Conf. (ACC)*, New Orleans, LA, 2021, pp. 3882–3889.
- [11] R. Cheng, G. Orosz, R. M. Murray, and J. W. Bagnell, “End-to-end safe reinforcement learning through barrier functions,” in *Proc. AAAI Conf. Artif. Intell.*, Honolulu, HI, 2019, pp. 3387–3395.
- [12] M. Jankovic, “Robust control barrier functions for constrained stabilization of nonlinear systems,” *Automatica*, vol. 96, pp. 359–367, Oct. 2018.
- [13] A. Clark, “Control barrier functions for complete and incomplete information stochastic systems,” in *Proc. Amer. Control Conf. (ACC)*, New Orleans, LA, 2021, pp. 2928–2935.
- [14] A. Singletary, S. Kolathaya, and A. D. Ames, “Safety-critical kinematic control of robotic systems,” *IEEE Control Syst. Lett.*, vol. 6, pp. 139–144, 2022.
- [15] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*. Cambridge, MA, USA: MIT Press, 2006.
- [16] N. Srinivas, A. Krause, S. M. Kakade, and M. W. Seeger, “Information-theoretic regret bounds for Gaussian process optimization in the bandit setting,” *IEEE Trans. Inf. Theory*, vol. 58, no. 5, pp. 3250–3265, May 2012.
- [17] D. Helbing and P. Molnár, “Social force model for pedestrian dynamics,” *Phys. Rev. E*, vol. 51, no. 5, pp. 4282–4286, May 1995.
- [18] T. Salzmann, B. Ivanovic, P. Chakravarty, and M. Pavone, “Trajectron++: Dynamically-feasible probabilistic human trajectory forecasting with heterogeneous data,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Glasgow, UK, 2020, pp. 683–700.
- [19] Y. Yuan, X. Weng, Y. Ou, and K. Kitani, “AgentFormer: Agent-aware transformers for socio-temporal multi-agent forecasting,” in *Proc. Int. Conf. Comput. Vis. (ICCV)*, Montreal, Canada, 2021, pp. 9813–9823.
- [20] A. Gupta, J. Johnson, L. Fei-Fei, S. Savarese, and A. Alahi, “Social GAN: Socially acceptable trajectories with generative adversarial networks,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Salt Lake City, UT, 2018, pp. 2255–2264.
- [21] A. J. Taylor and A. D. Ames, “Adaptive safety with control barrier functions,” in *Proc. Amer. Control Conf. (ACC)*, Denver, CO, 2020, pp. 1399–1405.
- [22] V. Dhiman, M. J. Khojasteh, M. Franceschetti, and N. Atanasov, “Control barriers

- in Bayesian learning of system dynamics,” *IEEE Trans. Autom. Control*, vol. 66, no. 10, pp. 4800–4807, Oct. 2021.
- [23] A. Castañeda, J. Choi, B. Zhang, D. E. Tomlin, and K. Sreenath, “Pointwise feasibility of Gaussian process-based safety-critical control under model uncertainty,” in *Proc. IEEE Conf. Decision Control (CDC)*, Austin, TX, 2021, pp. 6762–6769.
- [24] J. Quiñonero-Candela and C. E. Rasmussen, “A unifying view of sparse approximate Gaussian process regression,” *J. Mach. Learn. Res.*, vol. 6, pp. 1939–1959, Dec. 2005.
- [25] E. Snelson and Z. Ghahramani, “Local and global sparse Gaussian process approximations,” in *Proc. Int. Conf. Artif. Intell. Stat. (AISTATS)*, San Juan, Puerto Rico, 2007.
- [26] S. Oh, S. Russell, and S. Sastry, “Markov chain Monte Carlo data association for multi-target tracking,” *IEEE Trans. Autom. Control*, vol. 54, no. 3, pp. 481–497, Mar. 2009.
- [27] A. Lerner, Y. Chrysanthou, and D. Lischinski, “Crowds by example,” *Comput. Graph. Forum*, vol. 26, no. 3, pp. 655–664, 2007.
- [28] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, “OSQP: An operator splitting solver for quadratic programs,” *Math. Program. Comput.*, vol. 12, no. 4, pp. 637–672, Dec. 2020.
- [29] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge, UK: Cambridge Univ. Press, 2004.
- [30] H. K. Khalil, *Nonlinear Systems*, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2002.
- [31] N. Srinivas, A. Krause, S. Kakade, and M. Seeger, “Gaussian process optimization in the bandit setting: No regret and experimental design,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, Haifa, Israel, 2010, pp. 1015–1022.